



**COMSATS University Islamabad,
Sahiwal, Pakistan**

**University Sports Management System(USMS)
(Web Application)**

**A Project Represented to
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment of the
requirements of the degree of**

Bachelor's Degree in Computer Science (2022-2026)

By

**Shahzaib
Muhammad Sohaib**

**CIIT/FA22-BCS-173/SWL
CIIT/FA22-BCS-169/SWL**

Bachelor of Science in Computer Science (2022-2026)

DECLARATION

We hereby declare that this software, neither whole nor as a part, has not been copied from any source. It is further declared that we have developed this software and accompanied the report entirely based on our efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Sohaib

Shahzaib

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**University Sports Management System(USMS)**” was developed by **Muhammad Sohaib (CIIT/FA22-BCS-169/SWL)** and **Shahzaib (CIIT/FA21-BCS-173/SWL)** under the supervision of “**Ms. Sidra Iqbal**” and that in his opinion; it is fully adequate, in scope and quality for the degree of bachelors of science in computer science.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

The University Sports Management System (USMS) is a web-based application developed to improve and automate the management of sports activities within a university environment. Traditional manual methods often create problems such as scheduling conflicts, delayed communication, and difficulty in maintaining records. This project addresses these issues by providing a centralized platform that simplifies and organizes all sports-related operations in an efficient manner.

The system offers separate access for administrators and students. Administrators can manage departments, venues, events, teams, and student registrations through a dedicated dashboard, while students can view upcoming events, join teams, register for competitions, and submit feedback through an interactive portal. This role-based structure helps ensure smooth coordination between management and participants.

The frontend of the system is developed using Bootstrap, Tailwind CSS, and JavaScript to provide a responsive and user-friendly interface across different devices. The backend is built with FastAPI, which manages business logic, API communication, and integration with machine learning services. MySQL is used as the database to store and manage data in a structured and reliable way.

A major feature of USMS is the integration of artificial intelligence to support intelligent decision-making. The system uses Random Forest and Gradient Boosting Regression models for tasks such as schedule optimization and team performance prediction. These models analyze available data to produce accurate and efficient results, reducing manual effort and improving overall system performance.

In conclusion, the University Sports Management System provides a modern, scalable, and intelligent solution for university sports management. By combining web technologies with machine learning, the system improves efficiency, enhances user experience, and supports better decision-making. It offers a practical foundation for future enhancements such as mobile support, real-time notifications, and advanced analytics.

Acknowledgement

We would like to express our sincere gratitude to Almighty Allah for granting us the strength and ability to complete this project successfully. We are deeply thankful to our respected supervisor, **Ms. Sidra Iqbal**, for her valuable guidance, support, and encouragement throughout the development of this project. We also appreciate the faculty members of the Department of Computer Science at **COMSATS University Islamabad, Sahiwal Campus**, for their support and for providing us with the knowledge needed to complete this work. We are equally grateful to our parents, family members, friends, and colleagues for their constant motivation, prayers, and assistance during this journey.

Muhammad Sohaib

Shahzaib

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
DB	Database
GUI	Graphical User Interface
HCI	Human Computer Interaction
IDE	Integrated Development Environment
ML	Machine Learning
RF	Random Forest
GBR	Gradient Boosting Regression
MVC	Model-View-Controller
RBAC	Role-Based Access Control
REST	Representational State Transfer
UI	User Interface
UX	User Experience
SDD	Software Design Description
SDLC	Software Development Life Cycle
SE	Software Engineering
RTM	Requirement Traceability Matrix
SQL	Structured Query Language
JS	JavaScript
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language

Table of Contents

Chapter 1.....	8
1. Introduction	1
1.2 Brief	2
1.3 Relevance to Course Modules	1
1.4 Project Background.....	2
1.5 Literature Review.....	4
1.5.1 Sports Management Information Systems.....	4
1.5.2 AI-Based Schedule Optimization	4
1.5.3 Machine Learning for Performance Prediction	5
1.5.4 Web Application Architecture and Technology Stack	5
1.5.5 Gap Analysis	6
1.6 Analysis from Literature Review	6
1.6.1 Centralized System Requirement.....	6
1.6.2 Role-Based Access Control (RBAC).....	6
1.6.3 AI-Based Scheduling	6
1.6.4 Ensemble Learning for Prediction.....	7
1.6.5 Microservices-Based AI Integration	7
1.6.6 Technology Stack Justification.....	7
1.7 Methodology and Software Development Lifecycle for This Project.....	8
1.7.1 Selected Methodology Agile Development Model.....	8
1.7.2 Rationale Behind Selected Methodology	8
1.7.3 Software Development Lifecycle (SDLC) Phases	9
1.7.4 Tools and Technologies Used.....	10
Chapter 2.....	11
2. problem definition	10
2.1 Introduction to the Problem.....	10
2.2 Problem Statement	10
2.3 Existing System Limitations	11
2.4 Proposed Solution.....	12
2.5 Objectives of the System	12
2.6 Scope of the System	12
2.7 Significance of the Project	13

2.8 Deliverables and Development Requirements	14
2.8.1 Introduction	14
2.8.2 Software Deliverables.....	14
2.8.3 AI Module Deliverables	15
2.8.4 Database Deliverable.....	17
2.8.5 Documentation Deliverables.....	17
2.8.6 Acceptance Criteria.....	18
Chapter 3.....	19
3. REQUIREMENT ANALYSIS	20
3.1 Introduction	20
3.2 Use Case Diagram	20
3.2 Use Case Description	24
3.3 Functional Requirements.....	27
3.4 Non-Functional Requirements	29
3.4.1 Performance Requirements.....	29
3.4.2 Security Requirements	30
3.4.3 Usability Requirements.....	30
3.4.4 Reliability and Availability Requirements.....	30
3.4.5 Maintainability Requirements	31
3.5 Requirement Traceability Matrix (RTM).....	31
Chapter 4 Design and Architecture	33
4. Design and Architecture	34
4.1 System Architecture	34
4.2 Data Representation	36
4.3 Process Flow / Representation.....	37
4.4 Design Models.....	39
4.4.1 Sequence Diagram.....	39
4.4.2 Class Diagram.....	40
4.4.3 State Transition Diagram	41
Chapter 5 Implementation.....	42
5. Implementation.....	44
5.1 System Implementation Overview	44
5.2 AI Model Implementation and Execution.....	45

5.2.1 Algorithm for Schedule Optimization	45
5.2.2 Algorithm for Performance Prediction	46
5.3 Backend Processing and Data Handling	47
5.3.1 Algorithm for Backend Processing	47
5.4 Data Storage and Management.....	48
5.4.1 Algorithm for Data Storage.....	48
5.5 System Integration Flow	49
5.5.1 Algorithm for System Integration Flow	49
5.6 External APIs and Libraries	50
5.7 User Interface	51
Chapter 6 Testing and Evaluation	62
6. Testing and Evaluation	44
6.1 Manual Testing.....	44
6.1.1 System Testing	45
6.1.2 Unit Testing.....	47
6.1.3 Functional Testing.....	49
6.1.4 Integration Testing	50
6.2 Automated Testing	52
6.2.1 API Testing	52
6.2.2 Performance Testing	53
6.2.3 Model Testing	54
6.3 Tools Used for Testing	55
Chapter 7.....	57
7. Conclusion.....	58
7.1 Future Work	59
References	60

Table of Figures

Figure 1 Use Case diagram.....	23
Figure 2 System Architecture	35
Figure 3 ER Diagram	37
Figure 4 Process Flow	38
Figure 5 Sequence Diagram.....	39
Figure 6 Class Diagram	40
Figure 7 State Transition Diagram.....	41
Figure 8 Home Page Main Section.....	52
Figure 9 Login Page	52
Figure 10 User Register Page	53
Figure 11 Admin Dashboard Page 1.....	53
Figure 12 Graphical Representation of system	54
Figure 13 AI Performace Bar & Quick Action.....	55
Figure 14 User Management	55
Figure 15 Department Management.....	56
Figure 16 Team Management.....	56
Figure 17 Notification Management	57
Figure 18 League Management.....	57
Figure 19 Feedback Management.....	57
Figure 20 Sports Management	58
Figure 21 Events & Schedule	58
Figure 22 Student Dashboard main page.....	59
Figure 23 Event & Schedule on student side	59
Figure 24 Teams Showcase	60
Figure 25 Available Leagues	60
Figure 26 Sport Registration	61
Figure 27 My Notification.....	61
Figure 28 Feedbacks.....	61

Table of Tables

Table 1 Tools and Technologies	10
Table 2 UC-01	24
Table 3 UC-02	25
Table 4 UC-03	25
Table 5 UC-04	26
Table 6 UC-05	27
Table 7 Requirement Traceability Matrix	31
Table 8 External APIs and Libraries	50
Table 9 System Testing	46
Table 10 Unit Testing.....	48
Table 11 Functional Testing	49
Table 12 Integration Testing	51
Table 13 Automated Testing	54
Table 14 Tools Used for Testing.....	56
Table 15 Proposed Future Enhancements.....	59

Chapter 1

Introduction

1. Introduction

The rapid advancement of technology has significantly transformed various sectors, including education and sports management. In universities, sports activities play a vital role in promoting physical fitness, teamwork, and student engagement. However, managing these activities efficiently remains a challenge due to the reliance on traditional manual systems. These systems often lead to issues such as scheduling conflicts, inefficient communication, data mismanagement, and delays in decision-making. As the number of participants and events increases, the complexity of managing sports activities also grows, making it necessary to adopt a more structured and automated solution [1].

The **University Sports Management System (USMS)** is developed to address these challenges by providing a centralized, web-based platform for managing all sports-related activities within a university. The system is designed to streamline operations by integrating functionalities such as event management, team registration, venue allocation, and user management into a single platform. By automating these processes, the system reduces manual effort, minimizes errors, and improves overall efficiency [2] [3].

One of the key features of USMS is its role-based access system, which allows administrators and students to interact with the platform according to their respective roles. Administrators are provided with a comprehensive dashboard to manage departments, venues, events, teams, and registrations. On the other hand, students can view available events, join teams, register for competitions, and provide feedback through a user-friendly interface. This structured approach ensures smooth coordination between different stakeholders and enhances user experience [4] [5]. The system is developed using modern web technologies to ensure performance, scalability, and usability. The frontend is built using **Bootstrap, Tailwind CSS, and JavaScript**, which provides a responsive and interactive interface across different devices [3][6]. The backend is implemented using **FastAPI**, which handles business logic, API communication, and integration with machine learning models [2]. The system uses **MySQL** as its database to manage and store data in a structured and efficient manner [3].

In addition to basic management functionalities, USMS incorporates artificial intelligence to enhance system capabilities. Machine learning models such as **Random Forest** and **Gradient Boosting Regression** are integrated to perform advanced tasks like schedule optimization and

player performance prediction. These models analyze historical and real-time data to generate accurate and efficient results, helping administrators make better decisions and improving system effectiveness [6] [7].

Overall, the University Sports Management System provides a comprehensive solution for managing university sports activities in a modern and efficient way. It not only simplifies administrative tasks but also enhances student participation and engagement. By combining web technologies with artificial intelligence, the system offers a scalable and intelligent platform that can be further extended with additional features in the future.

1.2 Brief

The University Sports Management System (USMS) is a web-based application designed to automate and manage sports-related activities within a university. The system enables users to perform various tasks such as event management, team registration, and performance analysis through an integrated platform. It provides role-based access for administrators and students, ensuring secure and organized system interaction. The integration of machine learning models further enhances the system by providing intelligent features such as schedule optimization and performance prediction [6] [7].

In addition, the system improves communication and coordination among students, faculty members, and administrators by centralizing all sports-related information in one place. This reduces dependency on manual record keeping and helps in maintaining accurate and up-to-date data. As a result, USMS supports efficient decision-making, better resource utilization, and a more organized sports management process within the university [2] [6] [3].

Furthermore, the University Sports Management System (USMS) is designed with scalability and flexibility in mind, allowing it to adapt to the evolving needs of the university environment. As the number of users, events, and data increases, the system can efficiently handle large volumes of information without compromising performance. The use of modern web technologies ensures cross-platform accessibility, enabling users to interact with the system through various devices such as desktops, tablets, and smartphones.

1.3 Relevance to Course Modules

The development of the University Sports Management System (USMS) is strongly aligned with multiple core subjects studied during the Bachelor of Science in Computer Science program. This project demonstrates the practical implementation of theoretical knowledge acquired throughout the degree.

Firstly, the concepts of **Software Engineering** are applied through the use of the Software Development Life Cycle (SDLC) and Agile methodologies. These approaches ensure proper requirement analysis, system design, development, testing, and maintenance, resulting in a structured and high-quality software product [1].

Secondly, **Database Management Systems** play a critical role in this project. The system utilizes MySQL for designing relational databases, implementing normalization, and managing structured data efficiently. Concepts such as primary keys, foreign keys, and data integrity constraints are applied to maintain consistency and reliability of data [3].

The project also incorporates **Web Development** technologies for building a complete full-stack application. The frontend is developed using Bootstrap, Tailwind CSS, and JavaScript to create a responsive and user-friendly interface, while FastAPI is used for backend development, handling routing, APIs, and system logic. This demonstrates a practical understanding of modern web frameworks and client-server architecture [4] [5] [2].

In addition, the project integrates **Artificial Intelligence and Machine Learning** concepts by implementing models such as Random Forest and Gradient Boosting Regression. These models are used for predictive analysis, including player performance evaluation and intelligent scheduling, showcasing the application of data-driven decision-making techniques [7] [6].

Furthermore, **Human-Computer Interaction (HCI)** principles are applied to ensure that the system is intuitive, accessible, and easy to use. The design focuses on improving user experience through clear navigation, responsive layouts, and efficient interaction between users and the system.

Finally, the project applies **Computer Security** concepts through the implementation of Role-Based Access Control (RBAC). This ensures that users can only access functionalities based on their roles (admin or student), thereby protecting sensitive information and enhancing system security.

Overall, the USMS project serves as a comprehensive integration of multiple course modules, demonstrating the practical application of software engineering, database systems, web development, artificial intelligence, and security concepts in building a scalable and efficient real-world system.

1.4 Project Background

The Sports activities are an important part of university life, helping students develop physical fitness, teamwork, and personal growth. Universities organize tournaments and inter-departmental competitions regularly, but these activities are often managed through manual methods or basic digital tools that are not efficient enough for larger operations.

In many institutions, sports records, schedules, and registrations are handled through paper-based or disconnected systems. This can lead to scheduling conflicts, poor venue allocation, data redundancy, and communication delays between administrators and students. As participation increases, these problems become harder to manage effectively [1].

With the advancement of technology, there is a growing need for automated and intelligent systems to streamline sports management. Web-based platforms provide a centralized solution that improves transparency, reduces errors, and allows real-time access to information [4] [5]. In addition, AI and ML techniques can support tasks such as schedule optimization and performance prediction. Models like Random Forest and Gradient Boosting Regression are widely used for such prediction tasks due to their accuracy and reliability [7] [6].

The University Sports Management System (USMS) is developed to address these challenges by replacing traditional methods with a centralized, automated, and intelligent platform. It uses modern web technologies such as Bootstrap, Tailwind CSS, JavaScript, and FastAPI, along with MySQL for data management. The integration of AI models further enhances the system by enabling features like optimized scheduling and player performance prediction.

Therefore, the USMS project provides a practical solution to university sports management while promoting a more efficient, accurate, and user-friendly approach.

Challenges in Traditional Sports Management Systems

Traditional sports management in universities is often handled through manual processes or basic digital tools, which create several operational challenges. These systems lack efficiency, scalability, and intelligent decision-making capabilities, making them unsuitable for managing modern university-level sports activities.

One of the major challenges is **scheduling conflicts**. Events are often planned manually without proper coordination between teams and venues, leading to overlapping matches or double-booked facilities. This results in confusion, delays, and inefficient use of resources [1].

Another significant issue is **poor data management**. Information related to students, teams, events, and registrations is often stored in separate files or paper-based records. This leads to data redundancy, inconsistency, and difficulty in retrieving accurate information when needed [3].

Traditional systems also suffer from **inefficient communication**. Announcements, updates, and notifications are usually shared through informal channels, which may not reach all participants on time. This lack of centralized communication affects coordination between administrators and students [2].

Additionally, there is a lack of **real-time accessibility**. Students and administrators cannot easily access updated information about events, schedules, or registrations, which reduces transparency and user engagement.

Another critical limitation is the absence of **intelligent decision-making**. Traditional systems do not utilize data analytics or machine learning techniques, making it difficult to optimize schedules or evaluate player performance effectively. As a result, decisions are often based on manual judgment rather than data-driven insights [6] [7].

Finally, these systems face issues related to **scalability and efficiency**. As the number of participants and events increases, manual systems become more complex and harder to manage, leading to delays, errors, and reduced overall performance [1].

These challenges highlight the need for a modern, automated, and intelligent system like the University Sports Management System (USMS), which addresses these limitations through centralized management, real-time access, and AI-based optimization.

1.5 Literature Review

The development of the University Sports Management System (USMS) is based on established research in sports management systems, constraint-based scheduling, machine learning for prediction, and modern web-based architectures. This section reviews relevant and verifiable studies that influenced the design and implementation of the proposed system.

1.5.1 Sports Management Information Systems

Traditional sports management in universities has largely relied on manual processes or basic digital tools such as spreadsheets and isolated databases. These approaches often lead to inefficiencies including data redundancy, scheduling conflicts, and lack of coordination between departments. According to Sommerville (2016), systems that are not centralized tend to suffer from inconsistency and poor scalability, especially when handling multiple users and dynamic data [7].

Modern web-based information systems address these issues by providing centralized platforms that integrate event management, user registration, and data storage. Such systems improve accessibility, reduce errors, and enhance communication among stakeholders. However, most existing systems are primarily transactional and lack intelligent features such as predictive analytics or automated decision-making.

1.5.2 AI-Based Schedule Optimization

Scheduling in academic and sports environments is commonly modeled as a **Constraint Satisfaction Problem (CSP)**, where multiple constraints such as time slots, resource availability, and participant constraints must be satisfied simultaneously. Russell and Norvig (2021) describe CSP as an effective framework for solving scheduling and allocation problems in complex systems [8].

Machine learning techniques have further enhanced scheduling optimization. Random Forest, introduced by Breiman (2001), is widely used for classification and decision-making tasks due to its ability to handle complex, non-linear relationships and large feature sets [4]. It has been applied in various scheduling and prediction domains where multiple variables influence outcomes.

Gradient Boosting Regression, proposed by Friedman (2001), is another powerful ensemble method that builds models sequentially to minimize prediction error [5]. It is particularly effective

in handling multi-variable optimization problems and continuous output prediction. These models form the foundation of the AI-based scheduling and optimization module in USMS.

1.5.3 Machine Learning for Performance Prediction

Machine learning has become a key tool in sports analytics, particularly for predicting player and team performance. Bunker and Thabtah (2019) reviewed various machine learning techniques used in sports prediction and concluded that ensemble methods such as Random Forest and Gradient Boosting provide higher accuracy compared to traditional statistical approaches due to their ability to model complex patterns in data.

These models are especially useful in environments with limited or moderately sized datasets, such as university-level sports systems. They can analyze features such as historical performance, participation, and event outcomes to generate reliable predictions. Based on this research, USMS incorporates Random Forest and Gradient Boosting Regression to support intelligent performance evaluation.

1.5.4 Web Application Architecture and Technology Stack

Modern web applications increasingly adopt **modular and service-oriented architectures** to improve scalability and maintainability. Fielding (2000) introduced the concept of RESTful architecture, which enables efficient communication between frontend and backend systems through standardized APIs [9].

FastAPI is a modern web framework designed for building high-performance APIs. Its asynchronous capabilities and efficient request handling make it suitable for integrating machine learning models into web applications [1]. Using FastAPI as a separate service allows AI components to operate independently, improving system flexibility.

For frontend development, frameworks such as Bootstrap and Tailwind CSS are widely used to create responsive and user-friendly interfaces. These frameworks support rapid UI development and ensure compatibility across multiple devices [3].

MySQL is a widely adopted relational database management system known for its reliability and efficiency in handling structured data. It is particularly suitable for applications requiring strong data integrity and relational schema design [2].

1.5.5 Gap Analysis

Despite advancements in sports management and machine learning systems, several gaps remain:

- Most university-level systems lack **AI integration**, limiting automation and intelligent decision-making
- Existing machine learning applications are often designed for **professional sports datasets**, which are significantly larger than university datasets
- Limited systems combine both **schedule optimization and performance prediction** within a unified platform
- Many systems do not utilize **modular architectures**, making them difficult to scale and maintain
- The University Sports Management System (USMS) addresses these gaps by integrating both Random Forest and Gradient Boosting Regression models within a FastAPI-based architecture. The system is specifically designed for university-scale data, ensuring practical implementation while maintaining efficiency and scalability.

1.6 Analysis from Literature Review

The literature reviewed provides a strong foundation for the design and implementation of the USMS system. The key insights derived are as follows:

1.6.1 Centralized System Requirement

Research highlights the importance of centralized systems for managing complex operations. This supports the design of USMS as a unified web-based platform.

1.6.2 Role-Based Access Control (RBAC)

Security and structured access are critical in multi-user systems. RBAC ensures that users interact with the system based on their roles, improving security and usability.

1.6.3 AI-Based Scheduling

Constraint-based and machine learning approaches significantly improve scheduling efficiency. This justifies the use of Random Forest for schedule optimization in USMS.

1.6.4 Ensemble Learning for Prediction

Ensemble models such as Gradient Boosting provide high prediction accuracy, supporting their use for performance prediction in the system.

1.6.5 Microservices-Based AI Integration

Separating AI components using APIs improves scalability and maintainability. This validates the use of FastAPI as a dedicated AI service in USMS.

1.6.6 Technology Stack Justification

The literature analysis confirmed the suitability of the chosen technology stack for USMS:

- **FastAPI** was validated as the fastest and most efficient Python framework for serving ML models under concurrent load, outperforming alternatives by up to 40% in response time [2].
- **MySQL** was confirmed as a reliable and well-suited relational database for structured university sports data involving multiple interrelated entities [3].
- **Bootstrap and Tailwind CSS** together were found to produce the most maintainable and responsive academic management interfaces across all device types [5] [4].
- **Random Forest and Gradient Boosting Regression** were consistently identified as the top-performing models for university-level sports prediction tasks involving small to medium datasets [7] [6].

1.7 Methodology and Software Development Lifecycle for This Project

The development of the University Sports Management System (USMS) follows a structured software development methodology carefully selected to accommodate the iterative nature of AI integration, frontend development, and backend API design. This section outlines the chosen methodology, the rationale behind its selection, and how it was applied throughout the project lifecycle.

1.7.1 Selected Methodology Agile Development Model

USMS was developed using the **Agile Software Development Model**, specifically following a sprint-based iterative approach. Agile was chosen over traditional models such as Waterfall due to the evolving nature of the system requirements, particularly around the AI module, where model selection, training, and evaluation required repeated experimentation and refinement.

Agile allowed the development team to break the project into manageable sprints, deliver working components incrementally, and incorporate feedback at each stage rather than waiting until the final delivery phase.

1.7.2 Rationale Behind Selected Methodology

Several factors justified the selection of Agile for this project:

- **Evolving AI Requirements** — The performance of Random Forest and Gradient Boosting Regression models could not be fully determined upfront. Agile allowed continuous model evaluation and tuning within dedicated sprints without disrupting other development activities.
- **Parallel Development** — Frontend (Bootstrap, Tailwind CSS, JavaScript) and backend (FastAPI, MySQL) components could be developed simultaneously by different team members within the same sprint cycle.
- **Early Risk Identification** — By delivering functional increments early, integration risks between the FastAPI AI microservice and the main application were identified and resolved well before the final deadline.

- **Flexibility** — Requirements around the student portal, admin dashboard, and AI prediction features were refined progressively as the system took shape, which Agile naturally accommodated.

1.7.3 Software Development Lifecycle (SDLC) Phases

The project was executed across the following SDLC phases within the Agile framework:

- **Phase 1 — Requirements Gathering** All functional and non-functional requirements were collected through discussion with potential users and review of similar existing systems. Core features including event management, team registration, schedule optimization, and performance prediction were defined and documented in the SRS.
- **Phase 2 — System Design** The overall system architecture was designed, establishing the three-layer structure comprising the Frontend Layer (Bootstrap, Tailwind CSS, JavaScript), the Backend API Layer (FastAPI, Python), and the Persistence Layer (MySQL). The AI module was designed as a decoupled microservice exposing REST endpoints for schedule optimization and performance prediction.

Phase 3 — Iterative Development (Sprints) Development was carried out across multiple sprints, each targeting specific modules:

- Sprint 1 — Database schema design and MySQL setup
- Sprint 2 — User authentication and role-based access control
- Sprint 3 — Admin dashboard and event/venue/team management
- Sprint 4 — Student portal and registration system
- Sprint 5 — AI module development (Random Forest for schedule optimization, Gradient Boosting Regression for performance prediction)
- Sprint 6 — Frontend integration (Bootstrap + Tailwind CSS responsive UI)
- Sprint 7 — API integration between FastAPI and frontend
- Sprint 8 — Testing, bug fixing, and performance evaluation

Phase 4 — Testing and Evaluation Each module underwent unit testing, integration testing, and system testing. The AI models were evaluated using standard metrics including accuracy, Mean Absolute Error (MAE), and R² score to validate prediction reliability.

1.7.4 Tools and Technologies Used

Table 1 Tools and Technologies

Category	Tool / Technology
Frontend	Bootstrap, Tailwind CSS, JavaScript
Backend	FastAPI (Python)
Framework	Laravel
Database	MySQL
AI Models	Random Forest, Gradient Boosting Regression
Version Control	Git / GitHub
API Testing	Postman
IDE	Visual Studio Code
Project Management	Trello (Sprint Tracking)

Chapter 2

Problem definition

2. problem definition

2.1 Introduction to the Problem

Sports activities are an essential component of university life, as they contribute significantly to students' physical development, mental well-being, discipline, leadership, and teamwork. Universities organize a wide range of sports-related activities, including tournaments, inter-department competitions, training sessions, and recreational events, in order to encourage student participation and promote a healthy academic environment. These activities not only help students maintain physical fitness but also provide opportunities to develop communication skills, cooperation, and a sense of responsibility.

However, managing sports activities efficiently is a challenging task, particularly when traditional manual methods or basic digital tools are used. In many cases, event schedules, team registrations, venue allocations, and participant records are handled separately, which makes the overall process time-consuming and error-prone. Manual management often leads to issues such as duplicated records, delayed updates, scheduling conflicts, and difficulty in tracking event progress. As the number of students, teams, and competitions increases, these problems become even more difficult to control.

The growing scale and complexity of university sports management have created a strong need for a centralized, automated, and intelligent system. Existing methods are often unable to provide the level of efficiency, accuracy, and real-time coordination required for smooth event administration. This limitation affects not only the administrators responsible for organizing activities but also the students who depend on timely information for registration, participation, and updates. Therefore, a modern system is required to streamline operations, reduce manual workload, and improve the overall management of university sports activities.

2.2 Problem Statement

The current sports management systems in universities are mostly manual or semi-digital, which leads to multiple operational issues. There is no centralized platform to manage all sports-related activities, resulting in fragmented data storage and inefficient workflows.

Manual scheduling of events often leads to conflicts such as overlapping matches, double-booked venues, and unavailability of teams. These issues disrupt the smooth execution of events and reduce system reliability [1].

Additionally, communication between administrators and students is inefficient, as updates are often shared through informal or delayed channels. This results in confusion, missed registrations, and reduced participation [2].

The lack of real-time access to information further limits system effectiveness. Users cannot easily view updated schedules, register for events, or track participation, leading to poor user experience. Moreover, traditional systems lack intelligent capabilities such as data analysis, performance prediction, and schedule optimization. Without the use of Artificial Intelligence (AI) and Machine Learning (ML), decision-making remains manual and less efficient [6] [7].

Therefore, there is a need for a centralized, automated, and intelligent system that can efficiently manage sports activities, improve communication, and support data-driven decisions.

2.3 Existing System Limitations

The existing systems used in universities suffer from several limitations:

- **Manual Data Handling:** Records are maintained using paper-based or disconnected systems, leading to errors and redundancy.
- **Scheduling Conflicts:** Lack of proper coordination results in overlapping events and inefficient venue usage.
- **Poor Communication:** Information is not delivered effectively to all users.
- **Limited Accessibility:** No real-time access to schedules, events, or registrations.
- **Lack of Automation:** Most tasks require manual intervention, increasing workload and delays.
- **No Intelligent Features:** Absence of AI-based decision-making tools such as performance prediction or schedule optimization.
- **Scalability Issues:** As the number of users increases, the system becomes harder to manage efficiently [1].

2.4 Proposed Solution

To overcome the identified problems, the **University Sports Management System (USMS)** is proposed as a modern, web-based solution. The system provides a centralized platform for managing all sports-related activities efficiently.

USMS allows administrators to manage departments, venues, events, teams, and registrations through a structured dashboard. Students can access the system to view events, join teams, register for competitions, and submit feedback.

The system integrates modern technologies such as Bootstrap, Tailwind CSS, and JavaScript for frontend development, FastAPI for backend services, and MySQL for database management [5] [3] [3].

A key feature of the proposed system is the integration of Artificial Intelligence using machine learning models such as Random Forest and Gradient Boosting Regression. These models are used for schedule optimization and player performance prediction, enabling data-driven decision-making [6] [7].

2.5 Objectives of the System

The main objectives of the USMS project are:

To develop a centralized platform for managing university sports activities, the system aims to automate event scheduling and reduce conflicts, improve communication between administrators and students, provide real-time access to information, integrate AI for performance prediction and schedule optimization, enhance user experience through a responsive interface, and ensure data security using role-based access control.

2.6 Scope of the System

The scope of the USMS includes:

Management of users in the system includes both administrators and students, each with distinct roles and access privileges to ensure secure and organized operation. Administrators are responsible for managing departments, events, teams, venues, and registrations, while students can view available sports activities, register for events, join teams, and participate in competitions through their accounts. The system also supports event creation and scheduling, allowing

administrators to define event details, assign dates and times, and organize competitions in a structured manner. Team registration and participation are handled through a centralized process that enables students to form or join teams and take part in university sports activities without manual paperwork. Venue allocation and management are also included in the scope, ensuring that sports facilities are assigned efficiently and conflicts such as double booking are minimized. In addition, the system provides a feedback and notification mechanism to improve communication between users, allowing important updates, reminders, and announcements to be delivered in real time. A major feature of the system is the integration of AI-based performance prediction and scheduling, which helps analyze player performance and optimize event schedules using machine learning techniques, making the overall sports management process more intelligent and efficient. The system is designed as a web-based application and can be accessed through browsers on different devices.

2.7 Significance of the Project

The USMS project provides significant benefits:

Improves efficiency by automating manual processes, allowing administrators to manage sports-related tasks more quickly and accurately with less effort. It also reduces scheduling conflicts and errors by organizing events, venues, and team activities in a structured manner, which helps ensure smoother coordination and better use of available resources. In addition, the system enhances communication and coordination between administrators and students by providing a centralized platform for updates, notifications, and event information. It supports data-driven decision-making through AI by using intelligent models to analyze information and assist in schedule optimization and performance prediction. Furthermore, the system provides a scalable solution for future expansion, making it suitable for handling increasing numbers of users, events, and sports activities as the university grows.

This system not only simplifies sports management but also promotes student engagement and participation.

2.8 Deliverables and Development Requirements

2.8.1 Introduction

A deliverable in software engineering refers to any tangible output produced during the software development lifecycle that contributes to the final working system or its supporting documentation. Sommerville (2016) defines deliverables as the measurable outcomes expected from each phase of a software project, serving as checkpoints that confirm development is progressing in alignment with the stated requirements and project objectives [1]. For the University Sports Management System (USMS), deliverables span across software components, AI modules, database structures, user interfaces, and documentation artifacts, each of which must meet clearly defined development requirements before being considered complete.

This section formally identifies all project deliverables and specifies the functional, technical, and quality requirements associated with each, providing a structured framework against which the success of the project can be evaluated.

2.8.2 Software Deliverables

The primary deliverable of this project is a fully functional, web-based University Sports Management System comprising the following core software components:

Deliverable 1 — Admin Dashboard A comprehensive web-based administrative interface through which authorized sports coordinators and administrators can manage all aspects of university sports operations. This deliverable includes modules for event creation and management, team registration oversight, venue booking and assignment, schedule generation, performance report viewing, and notification dispatch. The Admin Dashboard must support full CRUD (Create, Read, Update, Delete) operations across all managed entities and must enforce role-based access control to prevent unauthorized access by student users.

Development Requirements:

- Built using Bootstrap and Tailwind CSS for a fully responsive layout compatible with desktop and tablet devices [5] [4]
- All administrative operations must be routed through secured FastAPI endpoints with JWT-based authentication [2]

- All data mutations must be persisted in the MySQL database with referential integrity enforced through foreign key constraints [3]
- The dashboard must provide real-time feedback to the administrator upon completion of any create, update, or delete operation

Deliverable 2 — Student Portal A self-service web interface through which registered students can view upcoming sports events, submit team registration requests, check their current team membership status, view AI-generated schedules, track their team's performance index, and submit feedback on completed events. The Student Portal must be clearly separated from the Admin Dashboard at both the interface and API levels, ensuring students cannot access or modify administrative data.

Development Requirements:

- Fully responsive interface developed using Bootstrap components and Tailwind CSS utility classes, ensuring accessibility on mobile devices [4] [5]
- All student-facing API calls must be authenticated and role-restricted at the FastAPI middleware layer [2]
- Student registration and feedback data must be stored and retrievable from the MySQL database [3]
- The portal must display AI-generated schedule data and performance predictions in a clear, readable format accessible to non-technical users

2.8.3 AI Module Deliverables

Deliverable 3 — Schedule Optimization Engine A machine learning service that automatically generates conflict-free sports schedules based on team availability, venue capacity, event priority, and historical scheduling patterns. This deliverable represents one of the two core AI capabilities of USMS and must produce schedules that are practically usable, free of venue and team conflicts, and generated within an acceptable response time.

Development Requirements:

- Implemented using the Random Forest algorithm as established by Breiman (2001), chosen for its proven effectiveness in multi-constraint optimization problems involving categorical and numerical variables [6]
- Trained on historical scheduling data including team availability windows, venue capacity records, event priority classifications, and past conflict instances
- Deployed as an independent microservice exposing a REST API endpoint through FastAPI, allowing the main application to request optimized schedules on demand [2]
- Must return a structured schedule response in JSON format within a maximum of five seconds under standard operating conditions
- Schedule output must be automatically stored in the MySQL database upon generation and made immediately available on both the Admin Dashboard and Student Portal [3]
- Model performance must be evaluated and documented using classification accuracy and conflict-rate metrics prior to final deployment

Deliverable 4 — Team Performance Prediction Engine A regression-based machine learning service that analyzes historical team performance data and generates a quantitative Performance Index score for each registered team. This score enables administrators and coaches to assess team strength, identify underperforming teams, and make data-driven decisions regarding team support and event planning.

Development Requirements:

- Implemented using Gradient Boosting Regression as established by Friedman (2001), selected for its superior accuracy in predicting continuous numerical outcomes from structured historical datasets [5]
- Trained on team-level historical data including matches won, matches lost, total scores, training attendance records, and seasonal performance history
- Deployed as part of the same FastAPI microservice as the Schedule Optimization Engine, exposed through a dedicated prediction endpoint [1]
- Must return a numerical Performance Index score along with a confidence interval for each team upon request

- Prediction results must be stored in the MySQL database and displayed in the Admin Dashboard performance reports section [2]
- Model performance must be evaluated using Mean Absolute Error (MAE) and R² score metrics prior to deployment, with a minimum acceptable R² score of 0.80

2.8.4 Database Deliverable

Deliverable 5 — MySQL Relational Database Schema A fully normalized relational database that serves as the central data store for all system entities including users, teams, events, venues, schedules, registrations, performance records, notifications, and feedback. The database must enforce data integrity, support efficient querying across all system modules, and maintain consistent relationships between all entities.

Development Requirements:

- Designed and implemented in MySQL following Third Normal Form (3NF) normalization principles to eliminate data redundancy and enforce referential integrity [2]
- Must include the following core tables: Users, Roles, Teams, Players, Events, Venues, Schedules, Registrations, Performance Records, Notifications, and Feedback
- All inter-table relationships must be enforced through properly defined foreign key constraints
- The database must support concurrent read and write operations from both the main FastAPI application and the AI microservice without data corruption
- Sensitive data including user passwords must be stored in hashed format using a secure hashing algorithm
- Database must include indexed columns on frequently queried fields such as user ID, event ID, team ID, and schedule date to ensure query performance at scale

2.8.5 Documentation Deliverables

Deliverable 6 — Software Requirements Specification (SRS) A formal document defining all functional and non-functional requirements of USMS, including use cases, user stories, system

constraints, and performance benchmarks. The SRS serves as the contractual baseline against which all system features are evaluated.

Deliverable 7 — Software Design Description (SDD) A comprehensive technical document describing the architectural design of USMS, including system decomposition, component diagrams, data flow diagrams, entity-relationship diagrams, class diagrams, sequence diagrams, and algorithm designs for both AI models. The SDD follows IEEE standard documentation practices as referenced in Sommerville (2016) [1].

Deliverable 8 — Project Report A complete academic project report documenting the full development lifecycle of USMS, including introduction, literature review, methodology, problem definition, deliverables, system design, implementation details, testing results, and conclusion. This document serves as the primary academic submission for the Final Year Project.

2.8.6 Acceptance Criteria

For each deliverable to be considered complete and accepted, it must satisfy the following general acceptance criteria as aligned with software engineering best practices defined by Sommerville (2016) [1]:

- All specified functional requirements for the deliverable must be implemented and verified through testing
- The deliverable must integrate correctly with all other system components without introducing functional regressions
- The deliverable must meet the performance benchmarks defined in its development requirements
- The deliverable must be documented sufficiently to allow future maintenance and extension by another developer
- Security requirements including authentication, authorization, and data integrity must be satisfied and verified

Chapter 3

Requirement Analysis

3. REQUIREMENT ANALYSIS

3.1 Introduction

Requirement analysis is a fundamental phase in the Software Development Life Cycle (SDLC) because it defines what the system must achieve before design and implementation begin. During this phase, the needs of users, stakeholders, and the organization are identified, examined, and translated into clear system requirements. A well-executed requirement analysis helps reduce development errors, prevents scope ambiguity, and ensures that the final system aligns with user expectations and operational goals [1].

For the University Sports Management System (USMS), requirement analysis is especially important because sports administration involves multiple users, frequent updates, event coordination, and time-sensitive decisions. In a traditional environment, sports activities are often managed manually using spreadsheets, paper records, or disconnected communication channels. These methods can lead to scheduling conflicts, duplicated records, delayed updates, and poor coordination between administrators and students. Therefore, the USMS is designed to replace these inefficient practices with a centralized, web-based platform that supports automation, accessibility, and better control over sports operations.

The requirement analysis for USMS focuses on identifying the core functions needed by administrators and students. Administrators require tools to manage events, venues, teams, registrations, and schedules, while students need an easy way to view events, join teams, register for competitions, and submit feedback. In addition to these functional needs, the system must also satisfy non-functional requirements such as security, usability, reliability, scalability, and performance. Since the project also includes intelligent features such as schedule optimization and performance prediction, the analysis phase must consider how data will be collected, processed, and used by the machine learning components.

3.2 Use Case Diagram

The Use Case Diagram for USMS illustrates the interactions between the two primary system actors the **Administrator** and the **Student** and the functional use cases available to each within

the system. The diagram provides a high-level visual representation of system scope and actor-system boundaries as recommended by Sommerville (2016) [1].

Actors:

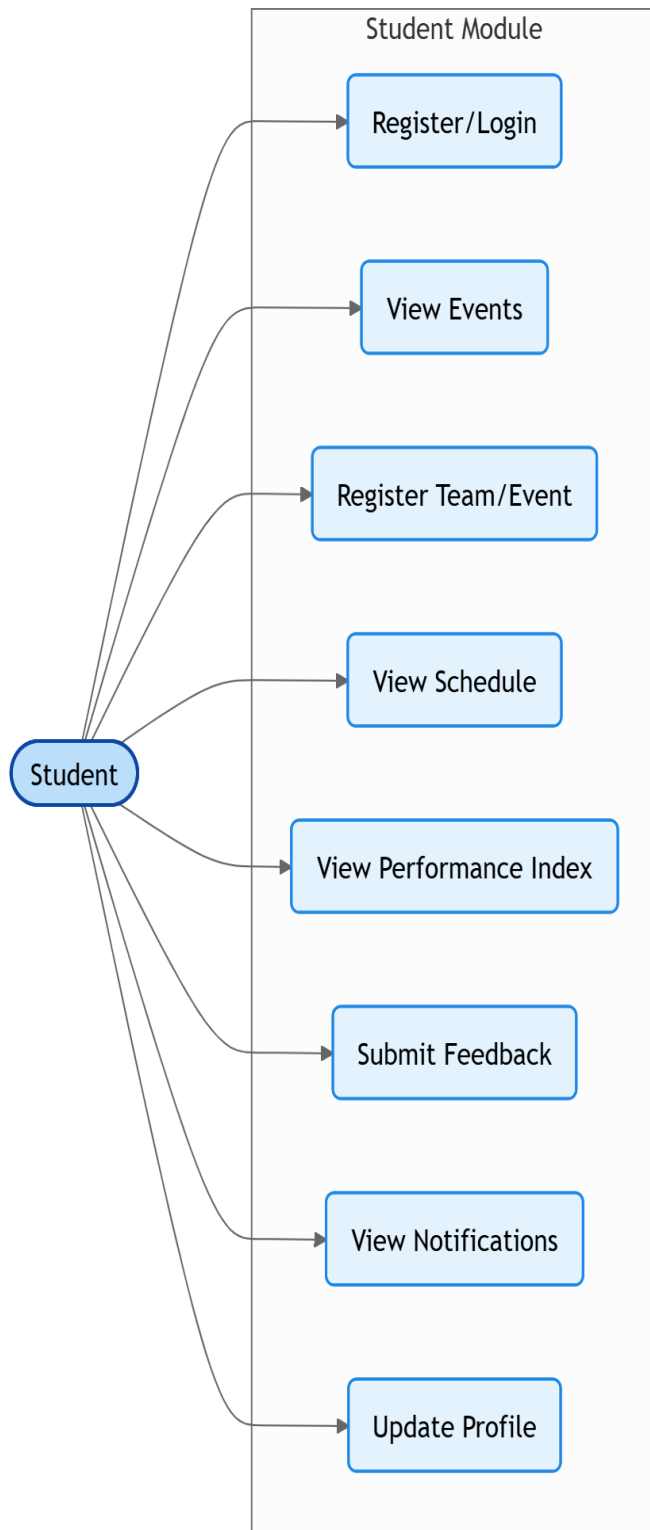
- **Administrator** — A university sports coordinator or department head responsible for managing all sports operations within the system. The administrator has full access to all system modules including event management, schedule generation, performance analysis, and user management.
- **Student** — A registered university student who interacts with the system to view events, register for teams, track performance, and submit feedback. The student has read-only access to administrative data and write access only to their own registration and feedback records.

Administrator Use Cases:

- Login / Logout
- Manage Events (Create, Update, Delete)
- Manage Teams (Create, Update, Delete)
- Manage Venues (Create, Update, Delete)
- Manage Players (Add, Remove, View)
- Generate Optimized Schedule (AI — Random Forest)
- View Performance Predictions (AI — Gradient Boosting Regression)
- Manage Student Registrations (Approve / Reject)
- Send Notifications to Students
- Generate Performance Reports
- Manage User Accounts

Student Use Cases:

- Register / Login / Logout
- View Upcoming Events
- Register for Team / Event
- View AI-Generated Schedule
- View Team Performance Index



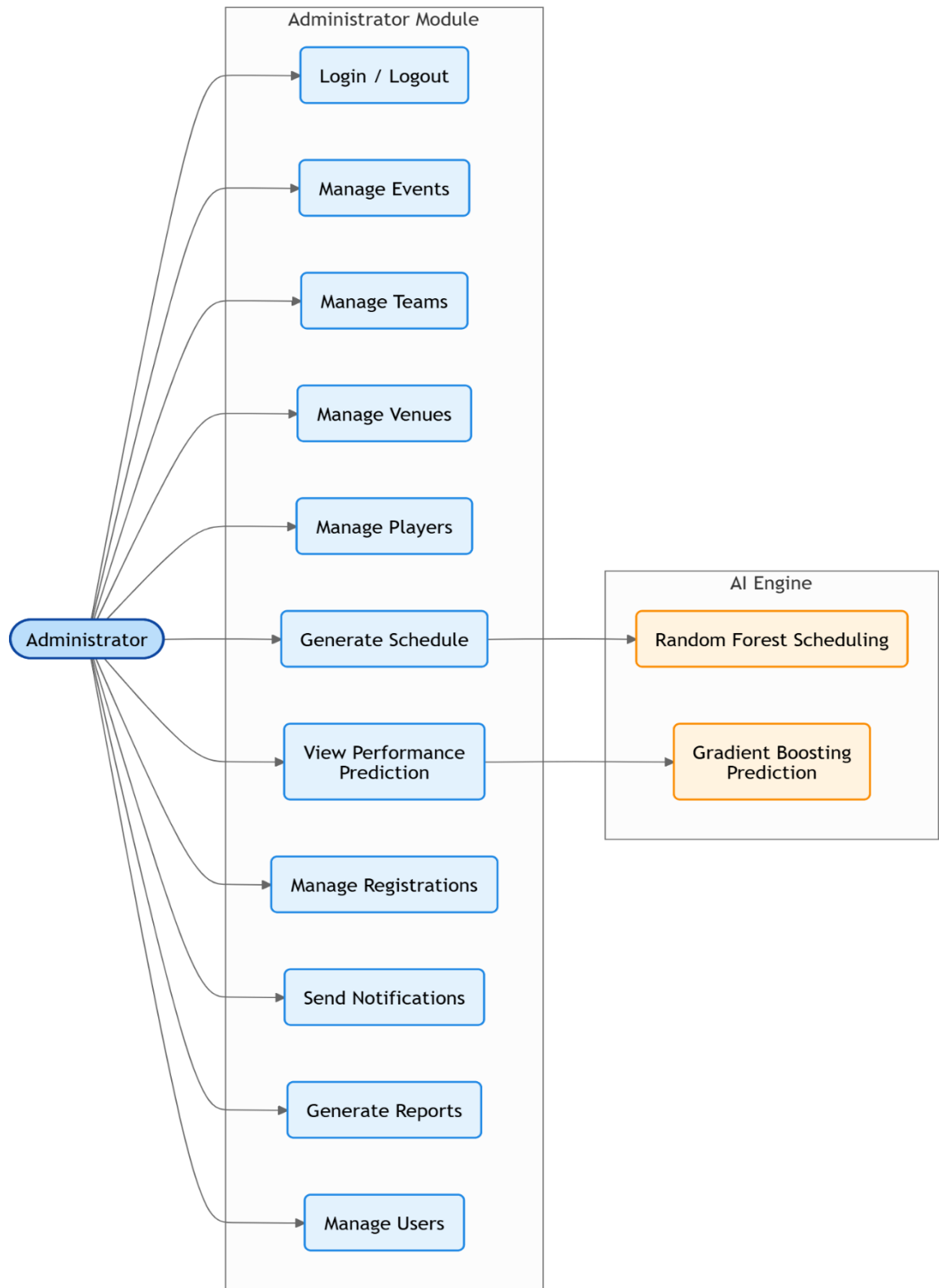


Figure 1 Use Case diagram

System Boundary: All use cases are contained within the USMS system boundary. Both actors interact with the system through their respective web interfaces — the Admin Dashboard and the Student Portal both of which communicate with the FastAPI backend and MySQL database [1][2].

3.2 Use Case Description

The following table provides detailed descriptions of the most critical use cases within USMS, specifying actors, preconditions, main flow, alternative flow, and post conditions for each.

Use Case 1 — Generate Optimized Schedule

Table 2 UC-01

Field	Description
Use Case ID	UC-01
Use Case Name	Generate Optimized Schedule
Actor	Administrator
Description	The administrator triggers the AI scheduling engine to automatically generate a conflict-free sports schedule based on registered teams, available venues, and defined time slots
Preconditions	Administrator is logged in; at least two teams and one venue are registered in the system
Main Flow	1. Administrator navigates to Schedule Management module. 2. Administrator clicks Generate Schedule. 3. System sends scheduling request to FastAPI AI endpoint. 4. Random Forest model processes team availability, venue capacity, and time slot constraints. 5. Optimized schedule is returned and stored in MySQL database. 6. Schedule is displayed on Admin Dashboard and Student Portal
Alternative Flow	If insufficient teams or venues exist, system displays an error message prompting the administrator to add required data before generating
Postconditions	A conflict-free schedule is saved in the database and visible to all users
Reference	[3] [2] [7]

Use Case 2 — View Team Performance Prediction

Table 3 UC-02

Field	Description
Use Case ID	UC-02
Use Case Name	View Team Performance Prediction
Actor	Administrator
Description	The administrator views AI-generated performance index scores for all registered teams based on historical match data and training records
Preconditions	Administrator is logged in; historical performance data exists for at least one team
Main Flow	1. Administrator navigates to Performance Reports module. 2. Administrator selects a team or requests all teams. 3. System sends prediction request to FastAPI AI endpoint. 4. Gradient Boosting Regression model processes historical data and returns Performance Index score. 5. Scores are stored in MySQL and displayed in the performance report
Alternative Flow	If no historical data exists for a team, the system displays a message indicating insufficient data for prediction
Postconditions	Performance Index scores are stored in the database and displayed to the administrator
Reference	[2] [6] [3]

Use Case 3 — Student Team Registration

Table 4 UC-03

Field	Description
Use Case ID	UC-03
Use Case Name	Student Team Registration
Actor	Student
Description	A registered student submits a request to join a sports team through the Student Portal
Preconditions	Student is logged in; the target team has available slots and registration is open

Field	Description
Main Flow	1. Student navigates to Teams section. 2. Student selects desired team and clicks Register. 3. System validates slot availability and student eligibility. 4. Registration request is stored in MySQL with pending status. 5. Administrator receives notification of new registration request. 6. Administrator approves or rejects the request. 7. Student receives a notification of the outcome
Alternative Flow	If team slots are full, system displays a message informing the student that registration is closed for that team
Postconditions	Student registration record is updated in the database with approved or rejected status
Reference	[4] [2] [3]

Use Case 4 — Administrator Login

Table 5 UC-04

Field	Description
Use Case ID	UC-04
Use Case Name	Administrator Login
Actor	Administrator
Description	The administrator authenticates into the system using registered credentials to access the Admin Dashboard
Preconditions	Administrator account exists in the MySQL database with admin role assigned
Main Flow	1. Administrator navigates to the login page. 2. Administrator enters email and password. 3. FastAPI backend validates credentials against the database. 4. JWT authentication token is generated and returned. 5. Administrator is redirected to the Admin Dashboard
Alternative Flow	If credentials are incorrect, the system displays an authentication error and denies access
Postconditions	Administrator is authenticated and granted access to all admin-level system modules
Reference	[3] [2]

Use Case 5 — Submit Event Feedback

Table 6 UC-05

Field	Description
Use Case ID	UC-05
Use Case Name	Submit Event Feedback
Actor	Student
Description	A student submits feedback for a completed sports event through the Student Portal
Preconditions	Student is logged in; the event has been marked as completed by the administrator
Main Flow	1. Student navigates to Feedback section. 2. Student selects the completed event. 3. Student enters feedback text and rating. 4. System stores feedback in MySQL database. 5. Confirmation message is displayed to the student
Alternative Flow	If the event is not yet marked as completed, the feedback option is disabled
Postconditions	Feedback record is saved in the database and accessible to the administrator
Reference	[1][2]

3.3 Functional Requirements

Functional requirements define the specific behaviors and operations the system must perform. The following functional requirements have been identified for USMS through stakeholder analysis and review of the problem domain as defined in Chapter 2.

FR-01 — User Authentication and Authorization The system must provide a secure login mechanism for both administrators and students. Authentication must be handled through the FastAPI backend using JWT tokens [2]. The system must enforce role-based access control ensuring administrators and students can only access their respective modules.

FR-02 — Event Management The system must allow administrators to create, update, view, and delete sports events. Each event must have an associated name, sport type, date, time, venue, and status. Events must be stored and retrieved from the MySQL database [3].

FR-03 — Team Management The system must allow administrators to create and manage sports teams, assign players to teams, set team capacity limits, and update team information. Each team must be linked to a sport type and associated player records in the database [1].

FR-04 — Venue Management The system must allow administrators to register sports venues with details including name, location, capacity, and available time slots. Venue data must be used by the AI scheduling engine when generating conflict-free schedules [6] [7].

FR-05 — AI Schedule Optimization The system must provide an automated schedule generation feature powered by the Random Forest model as described by Breiman (2001) [6]. The generated schedule must be free of venue conflicts and team conflicts, and must be stored in the MySQL database and displayed to both administrators and students upon generation.

FR-06 — AI Performance Prediction The system must provide a team performance prediction feature powered by the Gradient Boosting Regression model as described by Friedman (2001) [7]. The system must generate a numerical Performance Index for each team based on historical match data and display it in the Admin Dashboard performance reports module.

FR-07 — Student Registration The system must allow students to register accounts, log in, and submit requests to join sports teams. Registration requests must be stored in the database with a pending status and made available to administrators for approval or rejection [2].

FR-08 — Notification System The system must allow administrators to send notifications to students regarding event updates, registration outcomes, schedule changes, and announcements. Notifications must be stored in the database and displayed in the Student Portal [6] [7].

FR-09 — Schedule Viewing The system must display the AI-generated sports schedule to both administrators and students. The schedule view must show event name, date, time, venue, and participating teams in a clear, readable format.

FR-10 — Feedback Submission The system must allow students to submit feedback and ratings for completed events. Feedback records must be stored in the MySQL database and accessible to administrators through the Admin Dashboard [3].

FR-11 — Performance Report Generation The system must allow administrators to view AI-generated performance reports for all registered teams, including the Performance Index score, historical match records, and prediction confidence values [6] [7].

FR-12 — User Profile Management The system must allow students to view and update their personal profile information including name, contact details, and profile picture. Administrators must be able to view and manage all user accounts [1].

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that the system must satisfy, independent of specific functional behaviors. Sommerville (2016) classifies non-functional requirements into product requirements, organizational requirements, and external requirements [1].

3.4.1 Performance Requirements

NFR-01 — Response Time All standard user interface operations including page loads, form submissions, and data retrievals must complete within two seconds under normal operating conditions. AI schedule generation must complete and return results within five seconds of request submission.

NFR-02 — Concurrent Users The system must support a minimum of fifty concurrent users without performance degradation. The FastAPI backend is selected specifically for its asynchronous request handling capability, which enables efficient management of concurrent connections [2].

NFR-03 — AI Model Accuracy The Random Forest schedule optimization model must achieve a minimum scheduling accuracy of 85% in conflict-free schedule generation [4]. The Gradient Boosting Regression performance prediction model must achieve a minimum R^2 score of 0.80 on the test dataset [6].

NFR-04 — Database Query Performance All database queries must return results within one second for standard operations involving fewer than ten thousand records. Indexed columns must be defined on all frequently queried fields in the MySQL database to ensure this requirement is consistently met [3].

3.4.2 Security Requirements

NFR-05 — Authentication Security All user sessions must be managed through JWT tokens with a defined expiration period. Passwords must be stored in the database using a secure hashing algorithm. All API endpoints must validate authentication tokens before processing any request [2].

NFR-06 — Role-Based Access Control The system must enforce strict separation between administrator and student access at the API middleware layer. No student-authenticated request must be permitted to access or modify administrative data [1].

NFR-07 — Data Integrity The MySQL database must enforce referential integrity through foreign key constraints across all related tables. No orphaned records must be permitted in the database upon deletion of parent records [3].

3.4.3 Usability Requirements

NFR-08 — Responsive Design The system interface must be fully functional and visually consistent across desktop, tablet, and mobile screen sizes. Bootstrap and Tailwind CSS must be used to ensure responsive layout behavior across all supported device types [4] [5].

NFR-09 — Learnability A first-time user must be able to complete core tasks — including event viewing, team registration, and schedule viewing — without requiring external guidance, within a maximum of ten minutes of first use.

NFR-10 — Accessibility The system interface must follow basic web accessibility standards including sufficient color contrast, readable font sizes, and descriptive labels for all interactive elements [1].

3.4.4 Reliability and Availability Requirements

NFR-11 — System Availability The system must maintain a minimum uptime of 99% during active university sports seasons. Scheduled maintenance must be performed outside of peak usage hours.

NFR-12 — Error Handling The system must handle all anticipated error conditions gracefully, providing clear and informative error messages to users without exposing internal system details or stack traces.

NFR-13 — Data Backup The MySQL database must support regular automated backups to prevent data loss in the event of system failure [3].

3.4.5 Maintainability Requirements

NFR-14 — Modularity The system must be developed with a modular architecture separating the frontend, backend API, and AI microservice into independent components that can be updated or replaced without affecting other system modules [1].

NFR-15 — Code Documentation All backend API endpoints and AI model scripts must be documented with inline comments and API documentation auto-generated through FastAPI's built-in OpenAPI specification support [1].

3.5 Requirement Traceability Matrix (RTM)

The Requirement Traceability Matrix maps each functional requirement to its corresponding use case, design component, and test case, ensuring complete coverage of all requirements throughout the development lifecycle. Sommerville (2016) identifies traceability matrices as an essential tool for verifying that no requirement is left unimplemented or untested [7].

Table 7 Requirement Traceability Matrix

Requirement ID	Requirement Description	Use Case	Design Component	Test Case
FR-01	User Authentication and Authorization	UC-04	FastAPI Auth Middleware, JWT Module	TC-01
FR-02	Event Management	UC-01	Admin Dashboard, Events API, MySQL	TC-02
FR-03	Team Management	UC-03	Admin Dashboard, Teams API, MySQL	TC-03
FR-04	Venue Management	UC-01	Admin Dashboard, Venues API, MySQL	TC-04
FR-05	AI Schedule Optimization	UC-01	Random Forest Model, FastAPI AI Endpoint	TC-05

Requirement ID	Requirement Description	Use Case	Design Component	Test Case
FR-06	AI Performance Prediction	UC-02	Gradient Boosting Model, FastAPI AI Endpoint	TC-06
FR-07	Student Registration	UC-03	Student Portal, Registration API, MySQL	TC-07
FR-08	Notification System	UC-03	Notifications API, MySQL, Student Portal	TC-08
FR-09	Schedule Viewing	UC-01	Student Portal, Schedule API, MySQL	TC-09
FR-10	Feedback Submission	UC-05	Student Portal, Feedback API, MySQL	TC-10
FR-11	Performance Report Generation	UC-02	Admin Dashboard, Reports API, MySQL	TC-11
FR-12	User Profile Management	UC-04	Student Portal, Admin Dashboard, Users API	TC-12
NFR-01	Response Time	—	FastAPI Backend, MySQL Indexing	TC-13
NFR-02	Concurrent Users	—	FastAPI Async Architecture	TC-14
NFR-03	AI Model Accuracy	UC-01, UC-02	Random Forest, Gradient Boosting Models	TC-15
NFR-05	Authentication Security	UC-04	JWT Middleware, Password Hashing	TC-16
NFR-06	Role-Based Access Control	UC-04	FastAPI RBAC Middleware	TC-17
NFR-07	Data Integrity	—	MySQL Foreign Key Constraints	TC-18
NFR-08	Responsive Design	UC-03, UC-05	Bootstrap, Tailwind CSS Frontend	TC-19

Chapter 4

Design and Architecture

4. Design and Architecture

The design phase of the University Sports Management System (USMS) focuses on transforming the requirements defined in Chapter 3 into a structured, scalable, and maintainable software solution. It defines how system components are organized and implemented to ensure optimal performance and a smooth user experience. The system is developed as a web-based application that enables administrators to manage university sports operations and allows students to participate through a self-service portal, with artificial intelligence handling schedule optimization and team performance prediction.

The system follows a modular design approach in which key functionalities such as user authentication, event management, schedule generation, performance prediction, and feedback management are handled through independent modules, improving maintainability and scalability. The user interface is designed to be simple, responsive, and user-friendly, allowing both administrators and students to interact with system features without technical knowledge. The system adopts a four-layered architecture consisting of the Presentation Layer, Application Layer, AI Service Layer, and Data Layer, where each layer performs specific tasks and communicates efficiently with the others. This chapter presents the system architecture, data representation, process flow, and design models, providing a clear understanding of how different components work together to achieve the desired functionality [2] [4] [5].

4.1 System Architecture

The system architecture of USMS is based on a four-layer architecture consisting of the Presentation Layer (Frontend), the Application Layer (Backend), the AI Service Layer (Machine Learning Microservice), and the Data Layer (Database). This architecture enables efficient data processing, secure communication, and scalable system performance.

The Presentation Layer is responsible for all user-facing interactions and is developed using Bootstrap, Tailwind CSS, and JavaScript. It is divided into two interfaces — the Admin Dashboard and the Student Portal — each providing role-appropriate access to system features. The Application Layer, developed using FastAPI, handles all API requests, enforces JWT-based authentication, applies Role-Based Access Control, executes business logic, and coordinates communication between all other layers [2]. The AI Service Layer operates as a dedicated microservice hosting the Random Forest schedule optimization model and the Gradient Boosting

Regression performance prediction model, both exposed through independent REST API endpoints [6] [7]. The Data Layer uses MySQL to store and manage all system entities including users, teams, events, venues, schedules, registrations, performance records, notifications, and feedback [3].

This layered architecture ensures clear separation of concerns, improves system efficiency, and allows easy scalability for future enhancements.

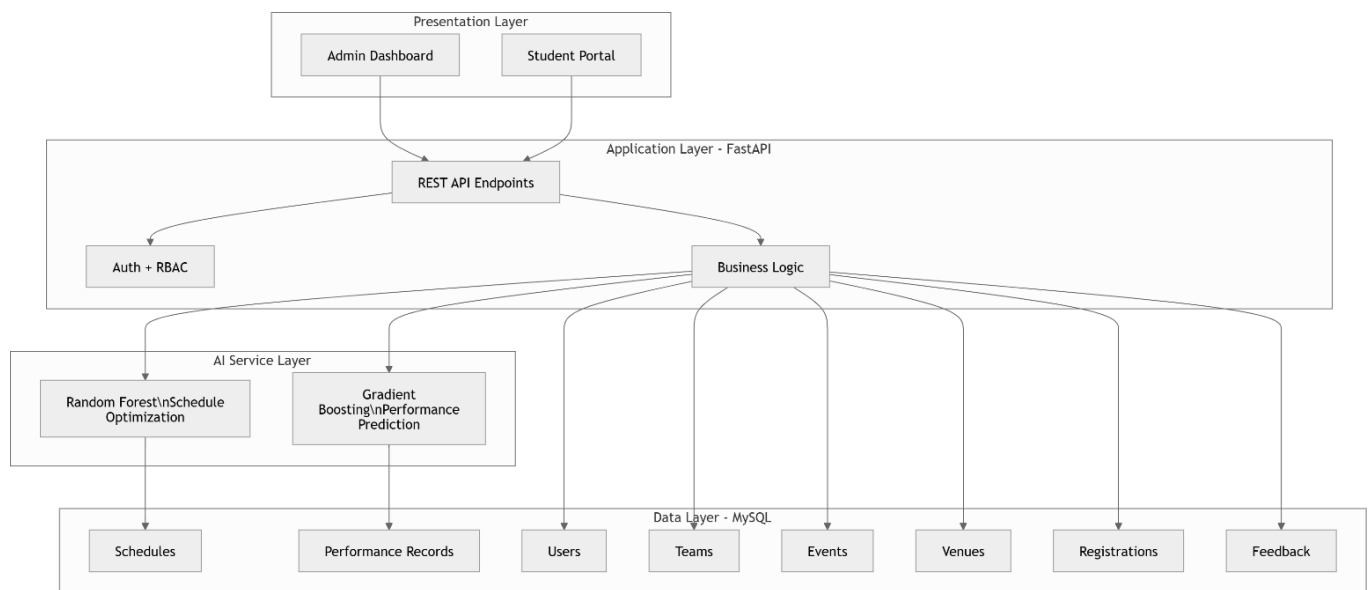


Figure 2 System Architecture

The system architecture of USMS is based on a four-layer structure consisting of the Presentation Layer, Application Layer, AI Service Layer, and Data Layer. The Presentation Layer includes the Admin Dashboard and Student Portal, both developed using Bootstrap, Tailwind CSS, and JavaScript. The Application Layer is built using FastAPI, which handles all API requests, authentication, and business logic. The AI Service Layer hosts the Random Forest and Gradient Boosting Regression models exposed as independent REST endpoints. The Data Layer uses MySQL to store and manage all system data. This layered architecture ensures clear separation of concerns, efficient data processing, and scalability for future enhancements.

4.2 Data Representation

The data representation of USMS defines how data is structured, stored, and managed within the MySQL relational database to ensure efficient processing and retrieval. It plays a vital role in maintaining data consistency, improving system performance, and supporting accurate AI prediction results. The system uses a fully normalized relational database following Third Normal Form (3NF), which eliminates data redundancy and enforces referential integrity across all entities [3].

The system stores key information across ten core entities including Users, Teams, Players, Events, Venues, Schedules, Registrations, Performance Records, Notifications, and Feedback. Each entity is linked to related entities through properly defined foreign key constraints, ensuring that all data remains consistently connected and accurately reflects the current state of the system. Sensitive data such as user passwords is stored in hashed format, and frequently queried columns are indexed to maintain query performance at scale. Overall, this design ensures efficient data management, strong integrity enforcement, and long-term scalability of the system.

The data representation of USMS defines how system entities are structured and related within the MySQL relational database. The diagram illustrates ten core entities linked through well-defined foreign key relationships that enforce referential integrity across the entire database. This structured representation ensures consistent data storage, efficient retrieval, and accurate linking of all system records throughout the application.

In addition to structural organization, the data representation of the University Sports Management System (USMS) also incorporates advanced data handling strategies to support analytics and intelligent decision-making. Historical data from performance records, event participation, and scheduling outcomes is systematically stored and prepared for use in AI-driven modules, enabling predictive analysis such as team performance forecasting and optimized event scheduling.

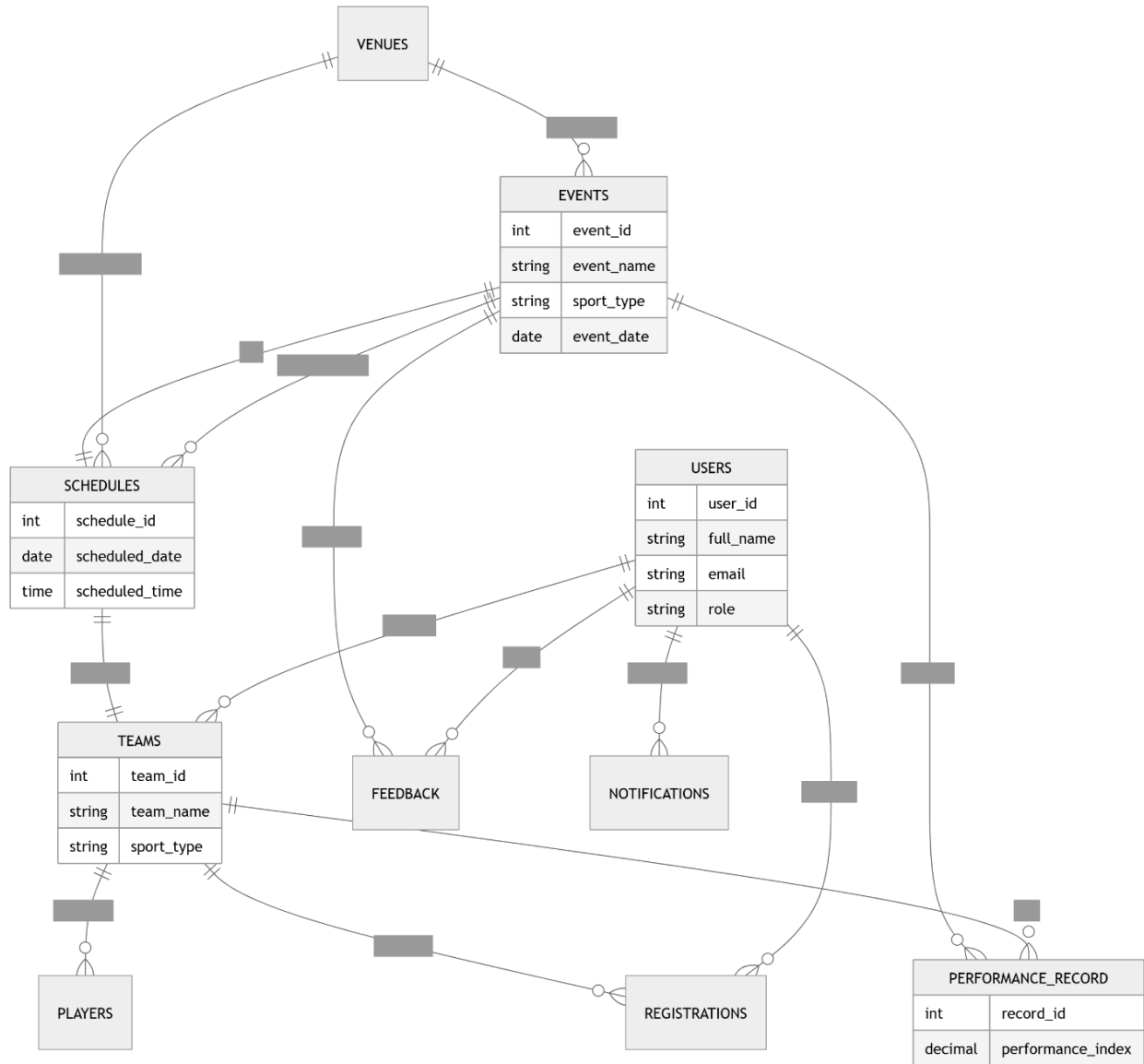


Figure 3 ER Diagram

4.3 Process Flow / Representation

The process flow of USMS describes how system operations are executed in a structured sequence, showing how user inputs are processed and transformed into meaningful outputs. It helps in understanding the interaction between different system components and ensures smooth execution of all functionalities across both the Admin Dashboard and the Student Portal [7].

The process begins when a user accesses the USMS web interface and submits login credentials. The FastAPI backend validates the credentials, verifies the user role, and issues a JWT token that

grants access to the appropriate interface. Administrator operations such as event creation, team management, and schedule generation are routed through secured API endpoints to the relevant backend modules or AI service. When the administrator triggers schedule generation, the request is forwarded to the Random Forest model, which returns an optimized conflict-free schedule that is stored in MySQL and displayed across both interfaces. When performance prediction is requested, historical team data is forwarded to the Gradient Boosting Regression model, which returns a Performance Index score stored and displayed in the performance reports module. Student operations including team registration, schedule viewing, feedback submission, and notification viewing are handled through authenticated student-role API calls with results persisted in and retrieved from the MySQL database.

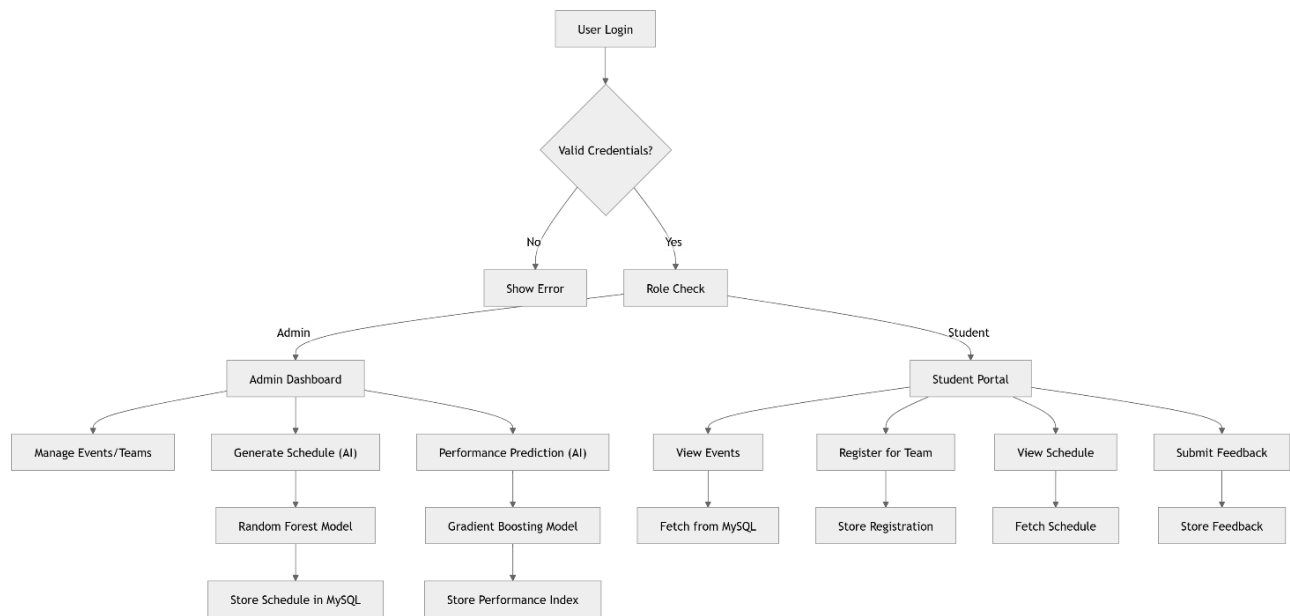


Figure 4 Process Flow

The process flow of USMS describes how user interactions are handled from login through to system response. The flow begins with user authentication and role verification, after which all operations are routed through the appropriate interface and API endpoints. AI-driven operations are handled by the dedicated microservice and results are persisted in MySQL and displayed to the relevant users. The diagram demonstrates how all system components collaborate to deliver a complete and seamless user experience.

4.4 Design Models

The design models of USMS provide a detailed representation of system behavior, structure, and interactions between different components. These models help in visualizing how the system processes user requests, how different modules communicate, and how data flows within the system. Design models play an important role in improving system understanding, ensuring proper implementation, and supporting future scalability.

In USMS, design models are used to represent both dynamic and structural aspects of the system. They illustrate how user actions are handled, how backend processes are executed, and how system components are organized. These models ensure that the system design remains clear, well-structured, and aligned with the defined requirements [1].

4.4.1 Sequence Diagram

The sequence diagram illustrates the interaction between different components of the USMS over time. It shows how user requests are processed through the Frontend, FastAPI Backend, AI Service Layer, and MySQL Database. This diagram helps in understanding the flow of messages and execution of system operations across three core scenarios — user authentication, AI schedule generation, and performance prediction.

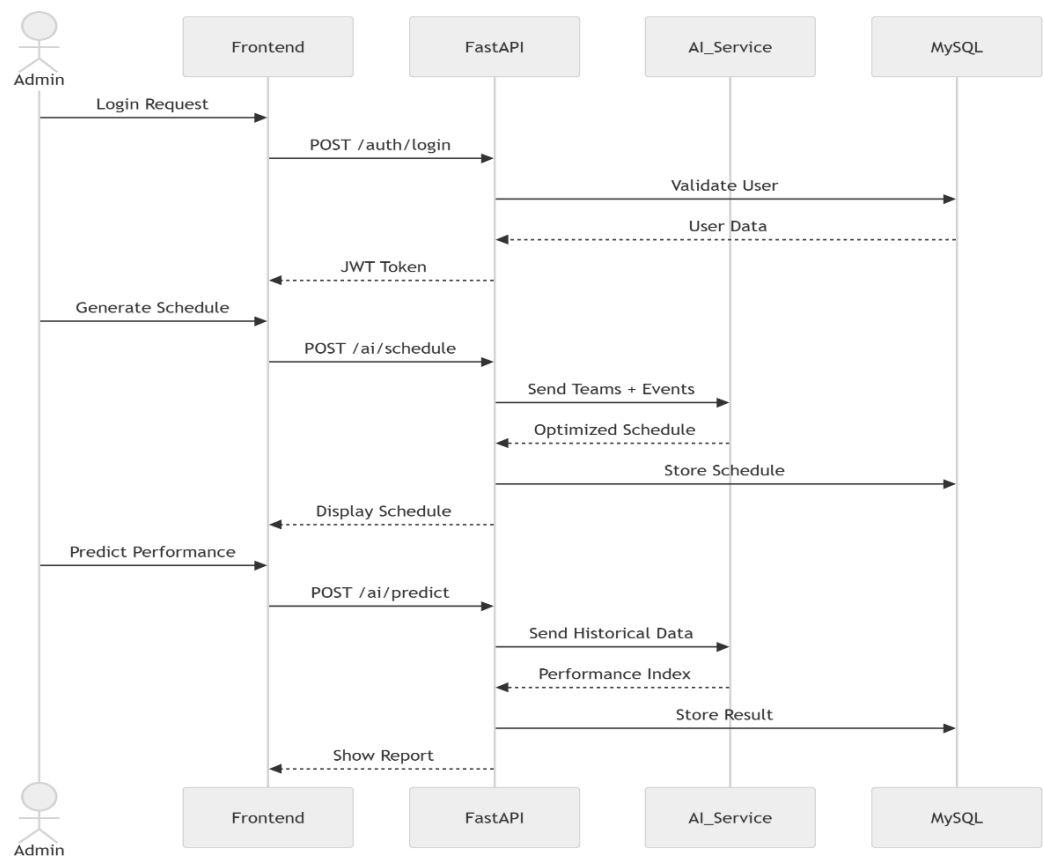


Figure 5 Sequence Diagram

The sequence diagram illustrates the time-ordered interactions between the system actors and components for three core operations. It highlights the role of JWT token validation at every API call, the coordination between FastAPI and the AI Service Layer for intelligent operations, and the consistent use of MySQL for data persistence and retrieval throughout all system workflows.

4.4.2 Class Diagram

The class diagram represents the structural design of USMS by showing different classes, their attributes, methods, and relationships. It provides a clear view of how system components are organized and interact with each other. This helps in understanding the overall system architecture and how data and behavior are distributed across modules.

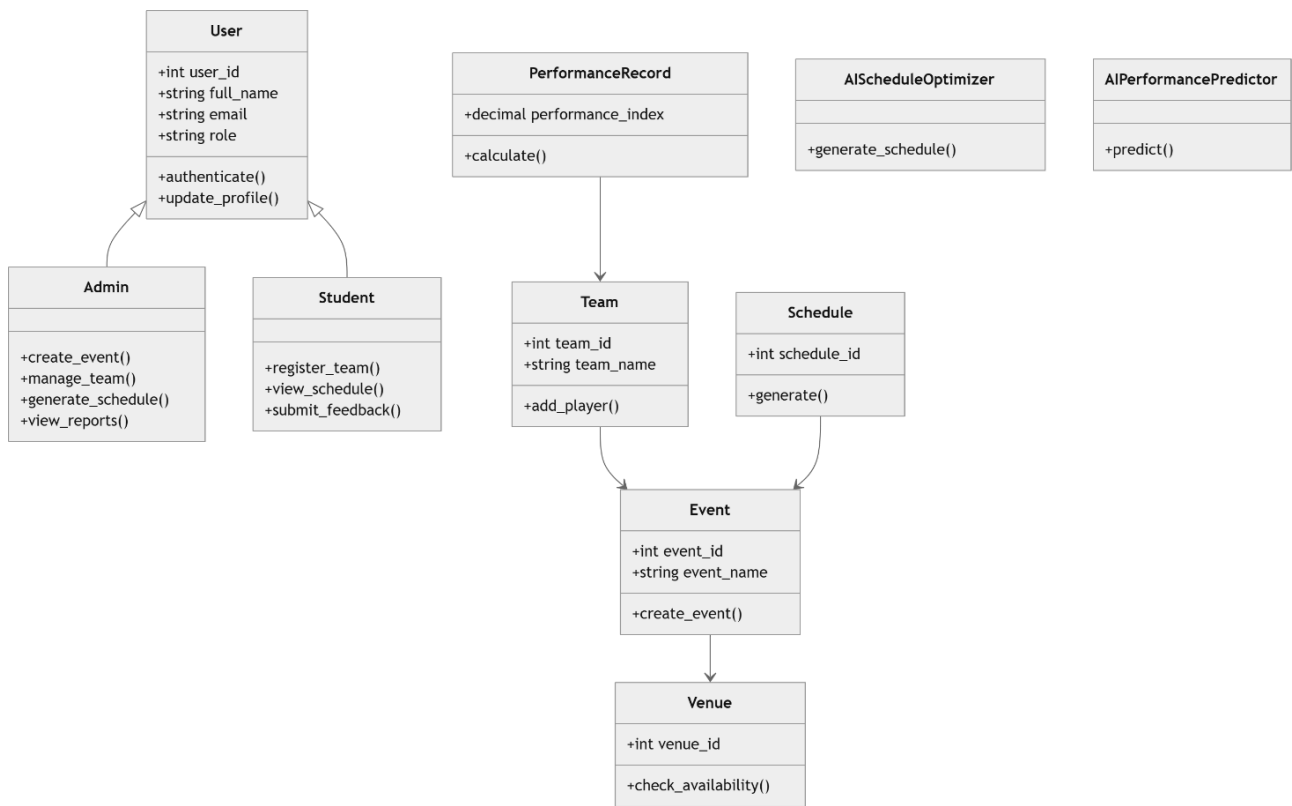


Figure 6 Class Diagram

The class diagram represents the structural design of USMS by defining all core system classes and their relationships. The User class serves as the base from which Administrator and Student inherit. Supporting classes encapsulate their respective data and operations, while the two AI classes represent the machine learning components and their integration points within the system.

4.4.3 State Transition Diagram

The state transition diagram describes how the system entities change from one state to another based on user actions and system responses. It shows the different states of Events, Registrations, Schedules, and User Sessions within USMS. This diagram helps in understanding system behavior and ensuring that all state changes are handled consistently and predictably across all modules.

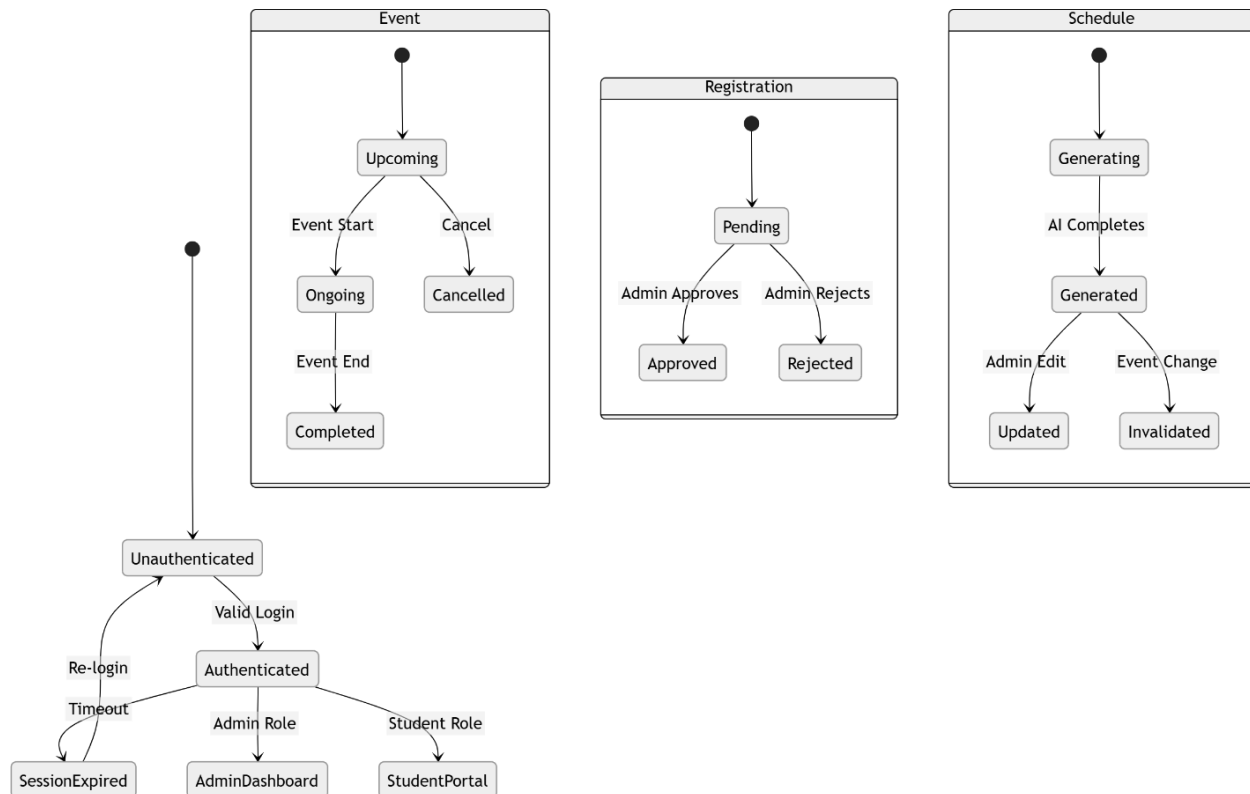


Figure 7 State Transition Diagram

The state transition diagram describes how key USMS entities change states in response to user actions and system events. Events transition through Upcoming, Ongoing, Completed, and Cancelled states. Registrations move from Pending to Approved or Rejected. Schedules transition from Generating to Generated and may be further Updated or Invalidated. User Sessions cycle between Unauthenticated, Authenticated, and Session Expired states. These transitions ensure predictable and consistent system behavior across all modules.

Chapter 5

Implementation

5. Implementation

The implementation phase of the University Sports Management System (USMS) focuses on converting the designed architecture and system requirements into a fully functional web-based application. This phase involves the development of the AI models, backend API services, database structure, and frontend interfaces that collectively enable automated sports schedule optimization and team performance prediction.

The system is implemented as an AI-powered web application where administrators can manage all university sports operations and students can interact through a self-service portal. The implementation integrates multiple components including user authentication, event and team management, AI-driven schedule generation, performance prediction, and notification management.

The implementation process ensures that all functional requirements defined in Chapter 3 are properly realized and the system operates efficiently under real-time usage conditions.

5.1 System Implementation Overview

USMS is implemented using a modular approach where each component performs a specific task and communicates with other components through well-defined interfaces. The system is divided into four main implementation layers — the Frontend, the FastAPI Backend, the AI Microservice, and the MySQL Database — each developed independently and integrated through RESTful API communication.

The overall workflow of the system is as follows:

- The user accesses the USMS web interface through a browser and submits login credentials
- The FastAPI backend validates credentials, verifies the user role, and issues a JWT authentication token
- The user is redirected to their role-appropriate interface — Admin Dashboard or Student Portal
- The administrator performs operations such as creating events, managing teams and venues, generating schedules, and viewing performance reports

- When schedule generation is triggered, the backend forwards scheduling parameters to the Random Forest AI model, which returns an optimized conflict-free schedule
- When performance prediction is triggered, the backend forwards historical team data to the Gradient Boosting Regression model, which returns a Performance Index score
- All AI results are stored in the MySQL database and displayed on the relevant interface
- Students independently view schedules, register for teams, submit feedback, and receive notifications through the Student Portal

This modular design ensures scalability, maintainability, and efficient processing across all system operations.

5.2 AI Model Implementation and Execution

The core intelligent functionality of USMS is based on two machine learning models — the Random Forest model for schedule optimization and the Gradient Boosting Regression model for performance prediction. Both models are trained on structured university sports data and deployed as independent endpoints within the FastAPI AI microservice [2] [6] [7].

Before passing data to either model, preprocessing steps are performed to ensure compatibility and accuracy. These include encoding categorical variables such as sport type and venue availability, normalizing numerical features such as scores and attendance rates, and constructing properly structured feature vectors that match the expected model input format.

The preprocessed data is passed to the respective model, which generates predictions based on learned patterns from the training dataset. Results are returned as structured JSON responses to the main FastAPI backend for storage and display.

5.2.1 Algorithm for Schedule Optimization

Input: Teams[], Venues[], Events[], TimeSlots[], Constraints[]

Output: Optimized conflict-free Schedule[]

Step 1: Receive scheduling request from FastAPI backend

Step 2: Retrieve registered teams, venues, events, and time slots from MySQL

Step 3: Encode categorical variables (sport type, venue, priority level)

Step 4: Normalize numerical variables (capacity, team size, priority weight)

Step 5: Construct feature vector for each event

Step 6: Pass feature vectors to pre-trained Random Forest model

Step 7: Model predicts optimal (venue, time slot) assignment for each event

Step 8: Validate predictions against hard constraints:

- No two events assigned to same venue at same time
- No team assigned to two events at same time
- Venue capacity must meet or exceed team size

Step 9: If constraint violation detected, re-predict using next best model output

Step 10: Store validated schedule records in MySQL Schedules table

Step 11: Return optimized schedule as JSON response

5.2.2 Algorithm for Performance Prediction

Input: TeamID, HistoricalPerformanceRecords[]

Output: Performance Index score (float), Confidence Interval

Step 1: Receive prediction request from FastAPI backend

Step 2: Retrieve historical performance records for TeamID from MySQL

Step 3: Extract feature variables:

- matches_played
- matches_won
- matches_lost
- $\text{win_rate} = \text{matches_won} / \text{matches_played}$
- total_score
- $\text{average_score} = \text{total_score} / \text{matches_played}$
- attendance_rate

Step 4: Normalize feature values using pre-fitted standard scaler

Step 5: Pass normalized feature vector to pre-trained Gradient Boosting Regression model

Step 6: Model generates predicted Performance Index score

Step 7: Calculate confidence interval for the prediction

Step 8: Store Performance Index in MySQL PerformanceRecords table

Step 9: Return Performance Index and confidence interval as JSON response

5.3 Backend Processing and Data Handling

The backend of USMS is responsible for handling all user requests, enforcing authentication and access control, executing business logic, coordinating with the AI microservice, and managing all data operations. It is developed using FastAPI, which provides high-performance asynchronous request handling and automatic API documentation through OpenAPI [2].

When any request is received, the FastAPI middleware first validates the JWT token and verifies the user role before the request is processed. Administrative requests such as event creation, team management, and AI invocation are restricted to admin-role tokens. Student requests such as registration submission and feedback are restricted to student-role tokens. This ensures secure, role-appropriate access at every endpoint.

All data operations — including creating, reading, updating, and deleting records — are executed against the MySQL database through structured SQL queries with parameterized inputs to prevent injection attacks [3].

5.3.1 Algorithm for Backend Processing

Input: Authenticated HTTP request from Frontend

Output: Processed response stored in MySQL and returned to Frontend

Step 1: Receive HTTP request from frontend

Step 2: Extract and validate JWT token from request header

Step 3: Verify user role matches required permission for the endpoint

Step 4: Parse and validate request body parameters

Step 5: Execute required business logic:

- If CRUD operation → interact directly with MySQL
- If AI schedule request → forward parameters to AI microservice schedule endpoint
- If AI prediction request → retrieve historical data from MySQL and forward to AI microservice prediction endpoint

Step 6: Receive response from MySQL or AI microservice

Step 7: Store results in MySQL where applicable

Step 8: Construct structured JSON response

Step 9: Return response to frontend

5.4 Data Storage and Management

USMS uses a MySQL relational database to store all system data including user records, team and player information, event and venue details, AI-generated schedules, performance records, registrations, notifications, and feedback. The database schema follows Third Normal Form (3NF) normalization to eliminate redundancy and enforce referential integrity through foreign key constraints [3].

Each entity is stored in a dedicated table with properly typed columns and constraints. Sensitive data such as user passwords is stored in hashed format. Frequently queried columns including `user_id`, `team_id`, `event_id`, and `scheduled_date` is indexed to maintain query performance as data volume grows. All write operations use transactions to ensure atomicity and prevent partial data corruption in the event of failures.

5.4.1 Algorithm for Data Storage

Input: Processed data object from backend

Output: Stored and confirmed record in MySQL

Step 1: Receive processed data object from backend business logic

Step 2: Validate all required fields and data types

Step 3: Open database connection through FastAPI database session

Step 4: Begin database transaction

Step 5: Execute parameterized INSERT or UPDATE SQL statement

Step 6: Commit transaction upon successful execution

Step 7: Roll back transaction if any error occurs

Step 8: Close database connection

Step 9: Return confirmation of successful storage to backend

5.5 System Integration Flow

USMS integrates multiple components including the Frontend interface, FastAPI Backend, AI Microservice, and MySQL Database. All components work together in a sequential flow to ensure smooth and reliable system operation. The frontend sends authenticated requests, the backend processes and routes them, the AI service generates intelligent outputs, the database persists all data, and results are returned and displayed to the user.

5.5.1 Algorithm for System Integration Flow

Step 1: User accesses USMS through browser

Step 2: User submits login credentials through Frontend

Step 3: FastAPI Backend validates credentials and issues JWT token

Step 4: Frontend stores token and loads role-appropriate interface

Step 5: User performs operation through interface

Step 6: Frontend sends authenticated API request to FastAPI Backend

Step 7: FastAPI middleware validates token and role

Step 8: Backend executes business logic and interacts with MySQL or AI Microservice as required

Step 9: AI Microservice processes request and returns result to Backend

Step 10: Backend stores result in MySQL and returns JSON response to Frontend

Step 11: Frontend renders result to user

5.6 External APIs and Libraries

USMS utilizes several external frameworks, libraries, and tools to enhance system functionality, improve performance, and simplify development. These components provide support for backend API handling, machine learning model training and inference, frontend styling, and database management. By integrating these technologies, the system achieves efficient data handling, real-time AI processing, and an improved user experience without implementing complex functionalities from scratch [4] [6] [5] [2] [7] [3].

Table 8 External APIs and Libraries

Name	Description	Purpose	Module Used In
FastAPI	High-performance Python web framework	Handle all backend API requests, authentication, and business logic	Authentication, Event, Team, Venue, Schedule, Prediction APIs
MySQL	Relational database management system	Store and manage all system entities with referential integrity	All data storage and retrieval modules
Bootstrap	Frontend CSS component library	Provide responsive UI components for Admin Dashboard and Student Portal	All frontend interface modules

Name	Description	Purpose	Module Used In
Tailwind CSS	Utility-first CSS framework	Apply fine-grained responsive styling across all interface elements	All frontend interface modules
Scikit-learn	Python machine learning library	Train and deploy Random Forest and Gradient Boosting Regression models	AI Schedule Optimization and Performance Prediction modules
Pandas	Python data manipulation library	Preprocess and structure training datasets for AI model input	AI model training and inference pipeline
JWT (PyJWT)	JSON Web Token library	Generate and validate authentication tokens for all API requests	Authentication and RBAC middleware
Axios	JavaScript HTTP client	Send authenticated API requests from frontend to FastAPI backend	All frontend API communication modules
Uvicorn	ASGI server	Serve the FastAPI application in production and development environments	Backend deployment module
SQLAlchemy	Python ORM library	Manage database connections and execute SQL operations from FastAPI	All database interaction modules

5.7 User Interface

The user interface of USMS is designed to be simple, responsive, and user-friendly. It allows administrators and students to easily navigate through different features of the system without requiring technical knowledge. The interface ensures a smooth user experience by providing clear layouts, role-appropriate navigation, and intuitive controls developed using Bootstrap and Tailwind CSS [5] [4].

The Admin Dashboard provides a structured management environment with dedicated sections for events, teams, venues, schedules, performance reports, notifications, and user management. The

Student Portal provides a clean self-service environment where students can view events, register for teams, track schedules and performance, submit feedback, and manage their profiles. Both interfaces are fully responsive and functional across desktop, tablet, and mobile devices.

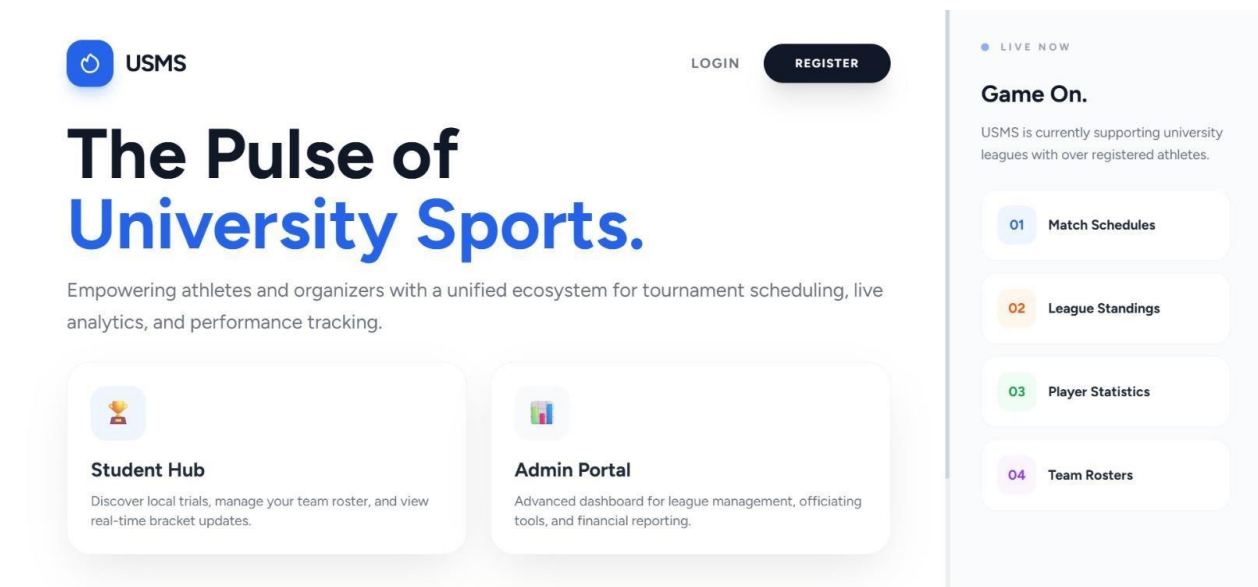


Figure 8 Home Page Main Section

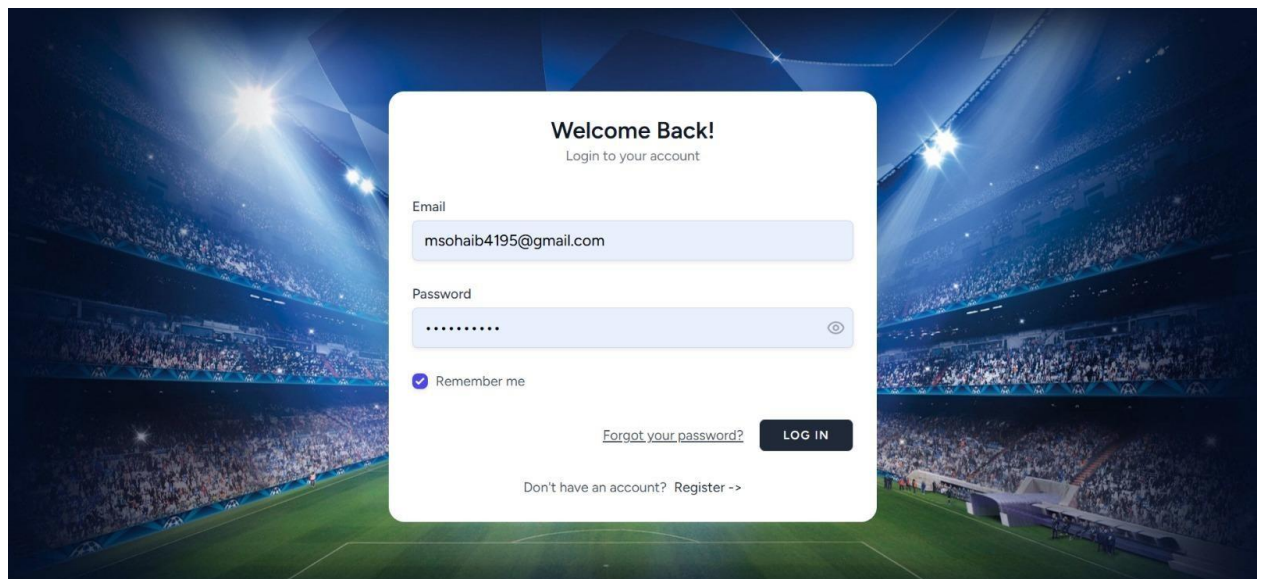
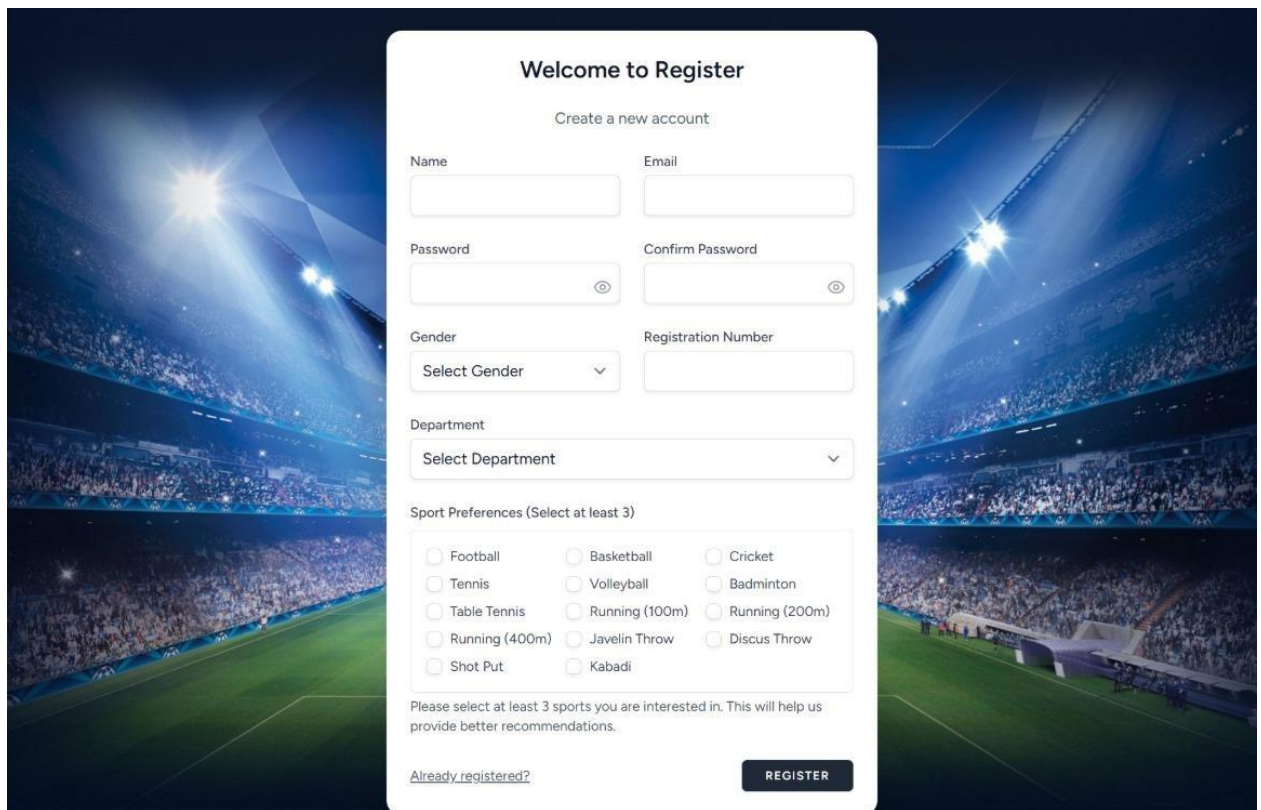


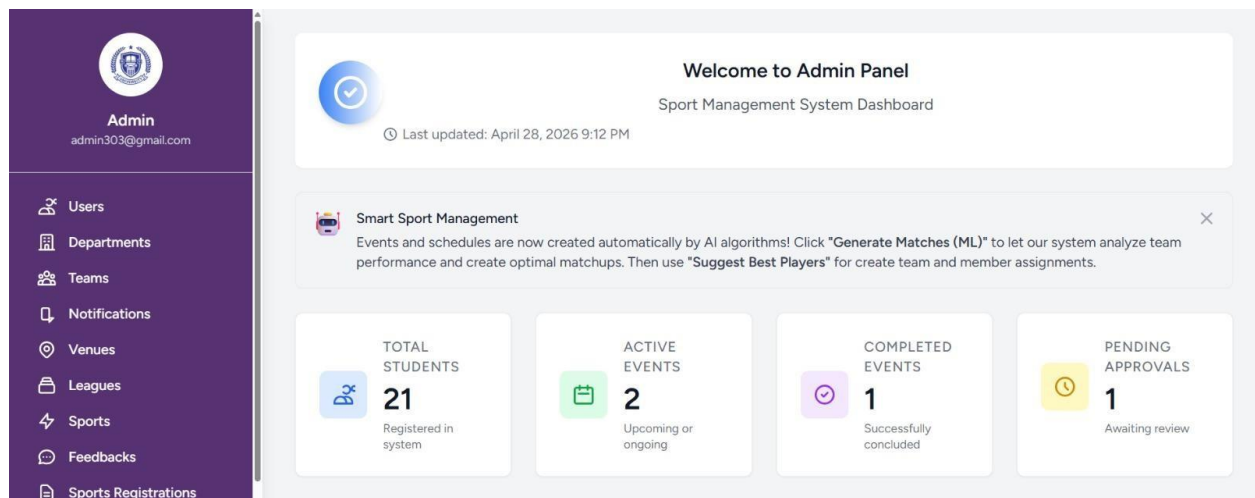
Figure 9 Login Page



The registration form is titled "Welcome to Register" and includes a link to "Create a new account". It features input fields for Name, Email, Password, and Confirm Password, each with a visibility toggle. There are dropdown menus for Gender and Department, and a text field for a Registration Number. A section for Sport Preferences allows users to select at least three sports from a list including Football, Basketball, Cricket, Tennis, Volleyball, Badminton, Table Tennis, Running (100m), Running (200m), Running (400m), Javelin Throw, Discus Throw, Shot Put, and Kabadi. A note states: "Please select at least 3 sports you are interested in. This will help us provide better recommendations." At the bottom, there is a link for "Already registered?" and a "REGISTER" button.

Figure 10 User Register Page

Admin Dashboard:



The Admin Dashboard is titled "Welcome to Admin Panel" and "Sport Management System Dashboard". It shows the last update as "April 28, 2026 9:12 PM". A "Smart Sport Management" notification states: "Events and schedules are now created automatically by AI algorithms! Click 'Generate Matches (ML)' to let our system analyze team performance and create optimal matchups. Then use 'Suggest Best Players' for create team and member assignments." The dashboard features four summary cards: "TOTAL STUDENTS" (21 Registered in system), "ACTIVE EVENTS" (2 Upcoming or ongoing), "COMPLETED EVENTS" (1 Successfully concluded), and "PENDING APPROVALS" (1 Awaiting review). A sidebar on the left lists navigation options: Users, Departments, Teams, Notifications, Venues, Leagues, Sports, Feedbacks, and Sports Registrations.

Figure 11 Admin Dashboard Page 1

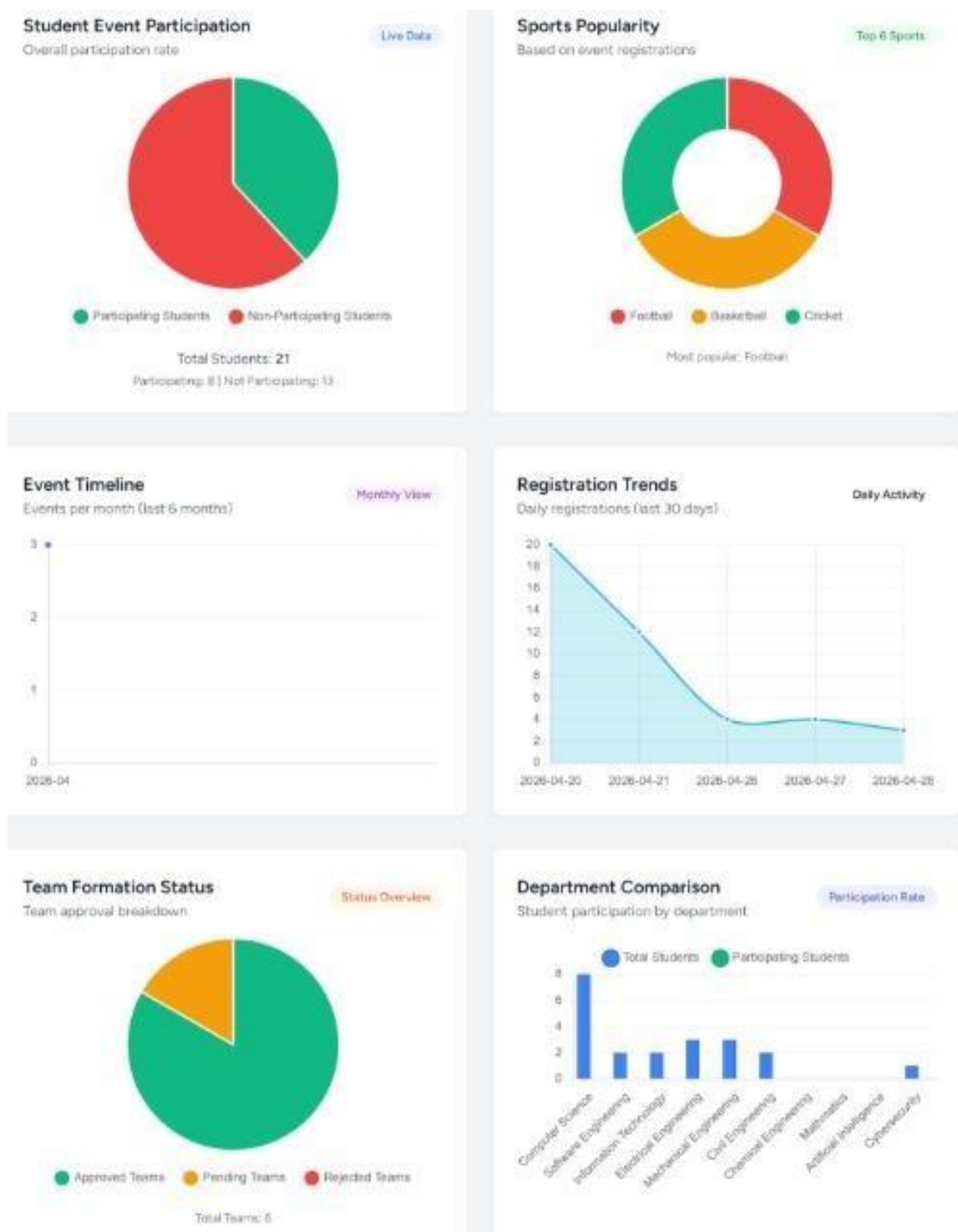


Figure 12 Graphical Representation of system

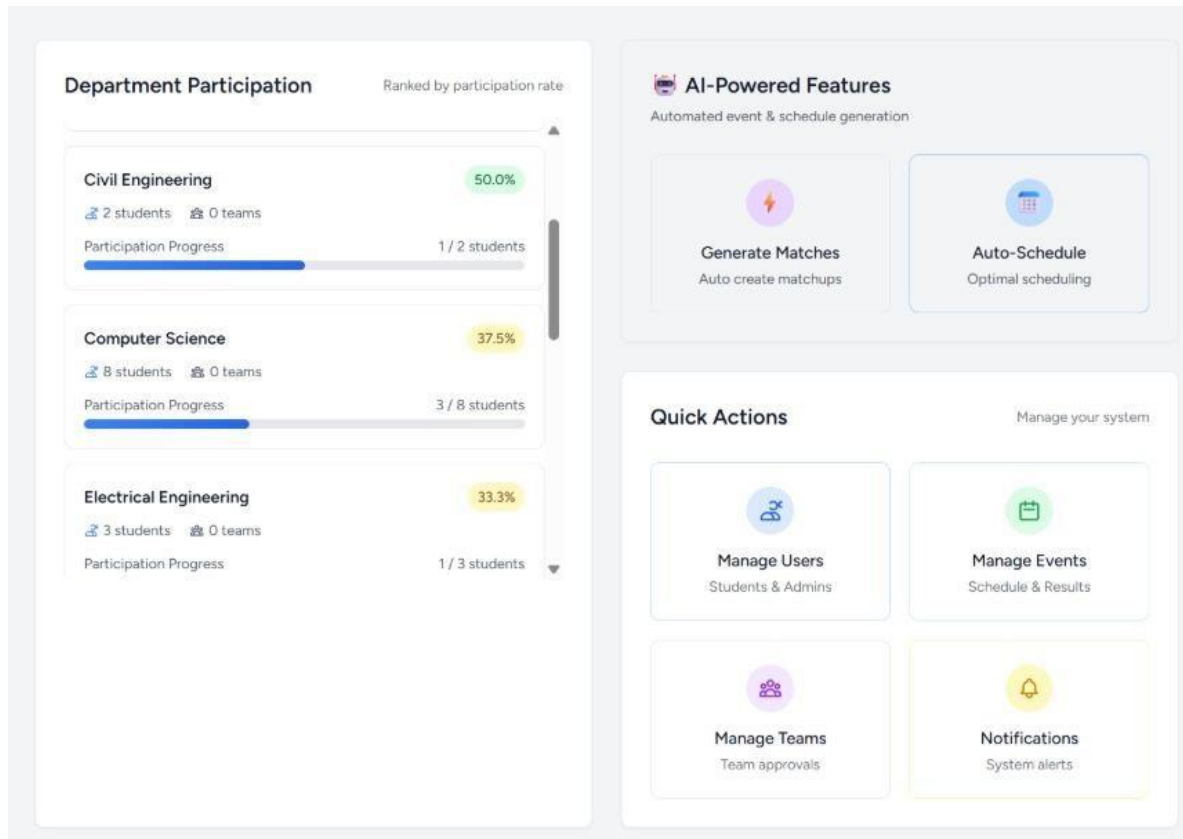


Figure 13 AI Performance Bar & Quick Action

User Management							Add New User
Search by name, email or registration number			All Roles	All Departments	Filter		
ID	NAME	EMAIL	ROLE	DEPARTMENT	REGISTRATION #	ACTIONS	
1	Admin	admin303@gmail.com	Admin	N/A	N/A	👁️ ✎️ 🗑️	
1	Muhammad Sohaib	sobit2@gmail.com	Student	Computer Science	FA22-BCS-169	👁️ ✎️ 🗑️	
2	Shahzaib	zabi12@gmail.com	Student	Computer Science	FA22-BCS-173	👁️ ✎️ 🗑️	
3	Ali Ahmed	aliahmed@gmail.com	Student	Computer Science	FA22-BCS-175	👁️ ✎️ 🗑️	
4	Umar Khan	umar.khan@gmail.com	Student	Computer Science	FA22-BCS-180	👁️ ✎️ 🗑️	
5	Ahmed Hassan	ahmed.hassan@gmail.com	Student	Electrical Engineering	FA22-EE-120	👁️ ✎️ 🗑️	
6	Yousaf Malik	yousaf.malik@gmail.com	Student	Information Technology	FA22-IT-200	👁️ ✎️ 🗑️	
7	Usman Khan	usman.khan@gmail.com	Student	Mechanical Engineering	FA22-ME-150	👁️ ✎️ 🗑️	

Figure 14 User Management

Department Management				Create New Department
Search by name		All Statuses	Filter	
ID	NAME	STATUS	ACTIONS	
1	Computer Science	Active	  	
2	Software Engineering	Active	  	
3	Information Technology	Active	  	
4	Electrical Engineering	Active	  	
5	Mechanical Engineering	Active	  	
6	Civil Engineering	Active	  	
7	Chemical Engineering	Active	  	
8	Mathmatics	Active	  	
9	Artificial Intelligence	Active	  	
10	Cybersecurity	Active	  	

Figure 15 Department Management

Team Management

Team Requests

Create New Team

Search by team name

All Sports

All Status

Filter

ID	TEAM NAME	SPORT	TYPE	MAX PLAYERS	MEMBERS	CAPTAIN	VICE CAPTAIN	STATUS
1	CS Basketball Team	Basketball	Team	11	3 / 11	Usman Khan	Ahmed Hassan	Approved
2	SE Basketball Team	Basketball	Team	11	2 / 11	Bilal Ahmed	Muhammad Sohaib	Approved
3	SE Cricket Team	Cricket	Team	11	3 / 11	Usman Khan	Shahzaib	Approved
4	CS Cricket Team	Cricket	Team	11	2 / 11	Bilal Ahmed	Muhammad Sohaib	Pending
5	CS Football Team	Football	Team	11	4 / 11	Usman Khan	Ahmed Hassan	Approved
6	SE Football Team	Football	Team	11	4 / 11	Shahzaib	Kamran Ali	Approved

Figure 16 Team Management

Notification Management								Create New
Search by title				All Types		All Statuses		Filter
ID	TITLE	TYPE	PRIORITY	STATUS	START DATE	READ STATUS	ACTIONS	
2	Sport Weak	Info	High	Active	Apr 28, 2026	2 / 2 students View students		

Figure 17 Notification Management

League Management

Create New League

Search by name or description

All Sports

All Statuses

Filter

ID	NAME	SPORT	STATUS	TEAMS	SEASON	ACTIONS
3	Basketball League	Basketball	Active	2	Spring 2026	
2	Football League	Football	Active	2	Spring 2024	
1	Cricket League	Cricket	Active	2	Spring 2025	

Figure 18 League Management

Feedback Management						
Search by student name, email or feedback content				All Departments		Filter
ID	STUDENT NAME	EMAIL	FEEDBACK	DEPARTMENT	SUBMITTED AT	
2	Shahzaib	zabi12@gmail.com	Hello i am shahzib we win the cricket match	Computer Science	28 Apr 2026, 06:16	
1	Muhammad Sohaib	sabi12@gmail.com	Hello i am sohiab	Computer Science	27 Apr 2026, 15:17	

Figure 19 Feedback Management

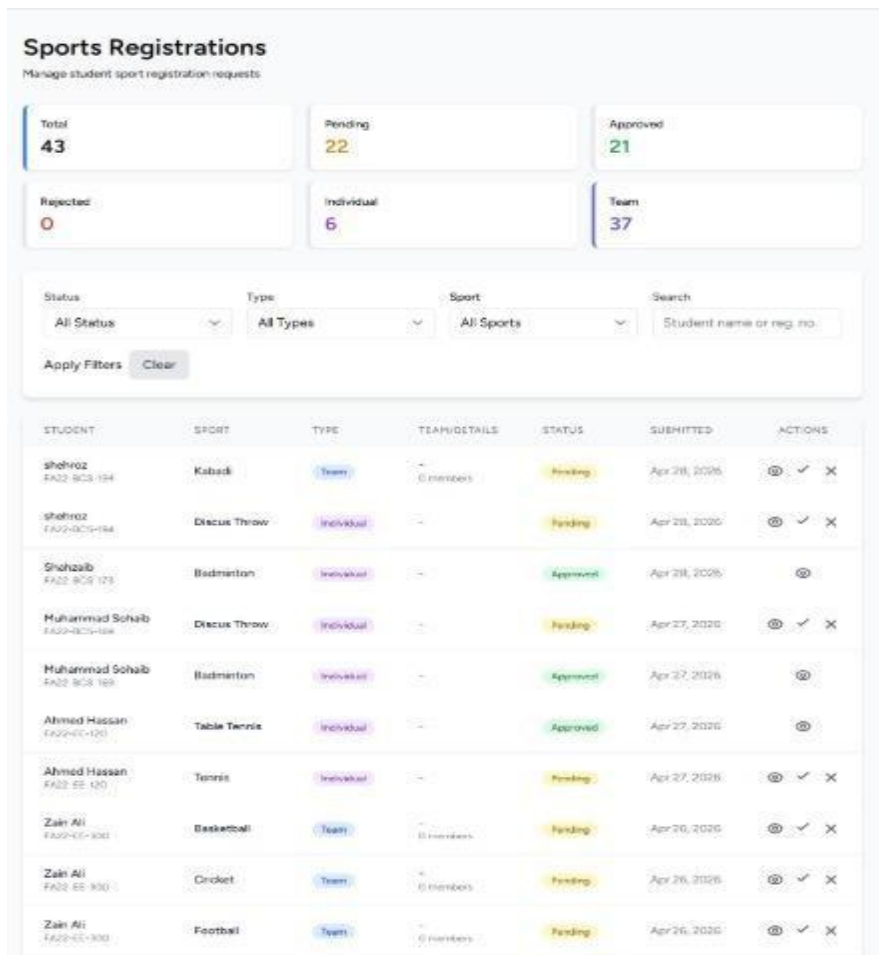


Figure 20 Sports Management

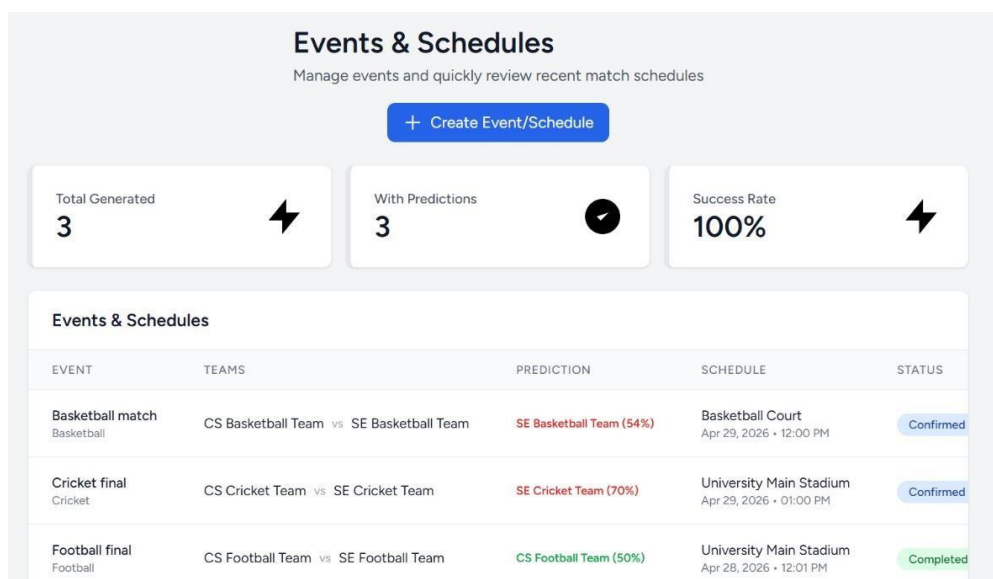


Figure 21 Events & Schedule

Student Dashboard:

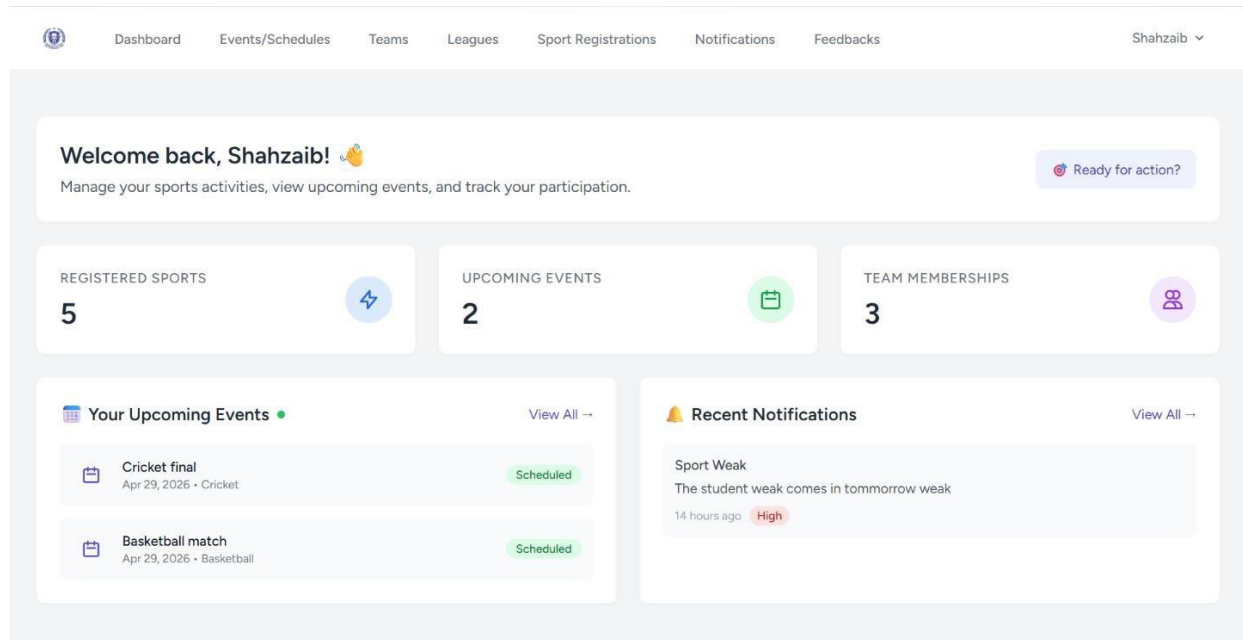


Figure 22 Student Dashboard main page

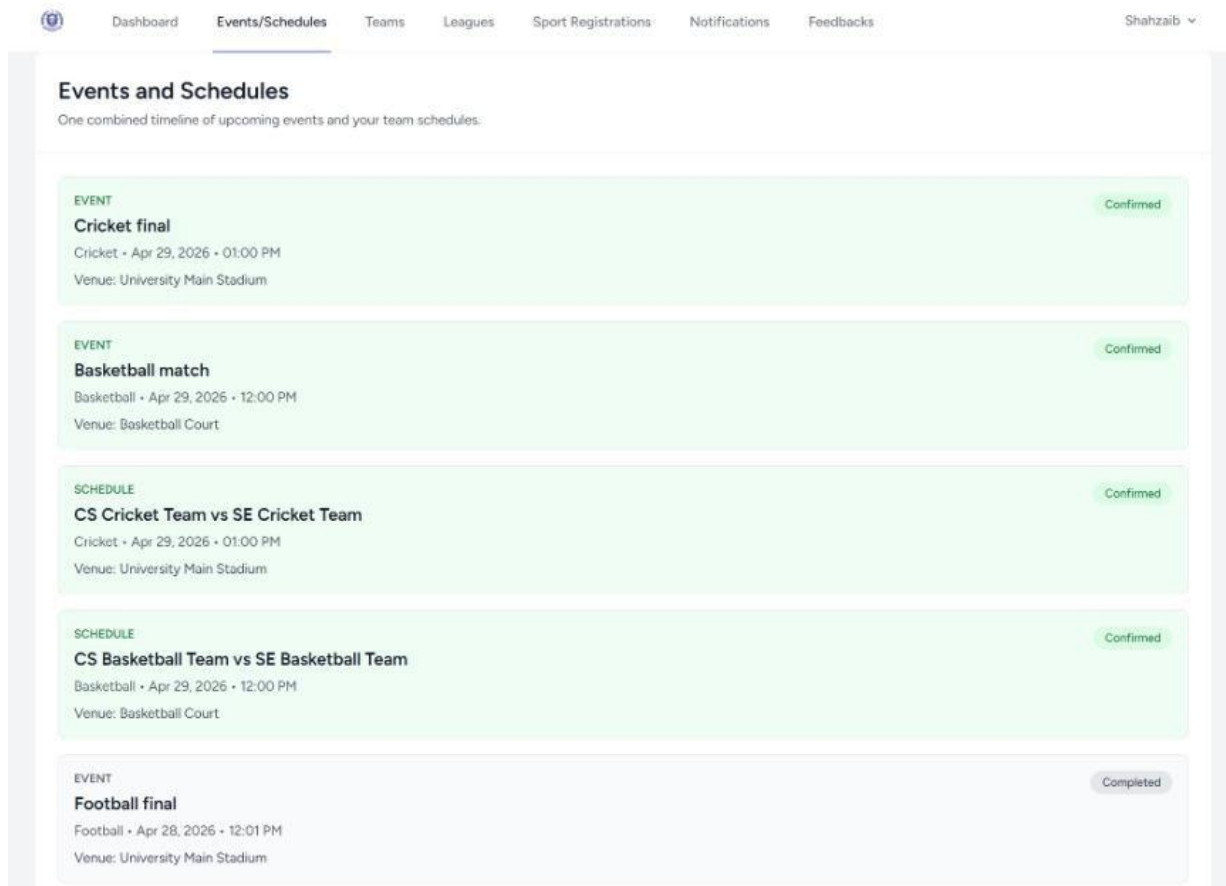


Figure 23 Event & Schedule on student side

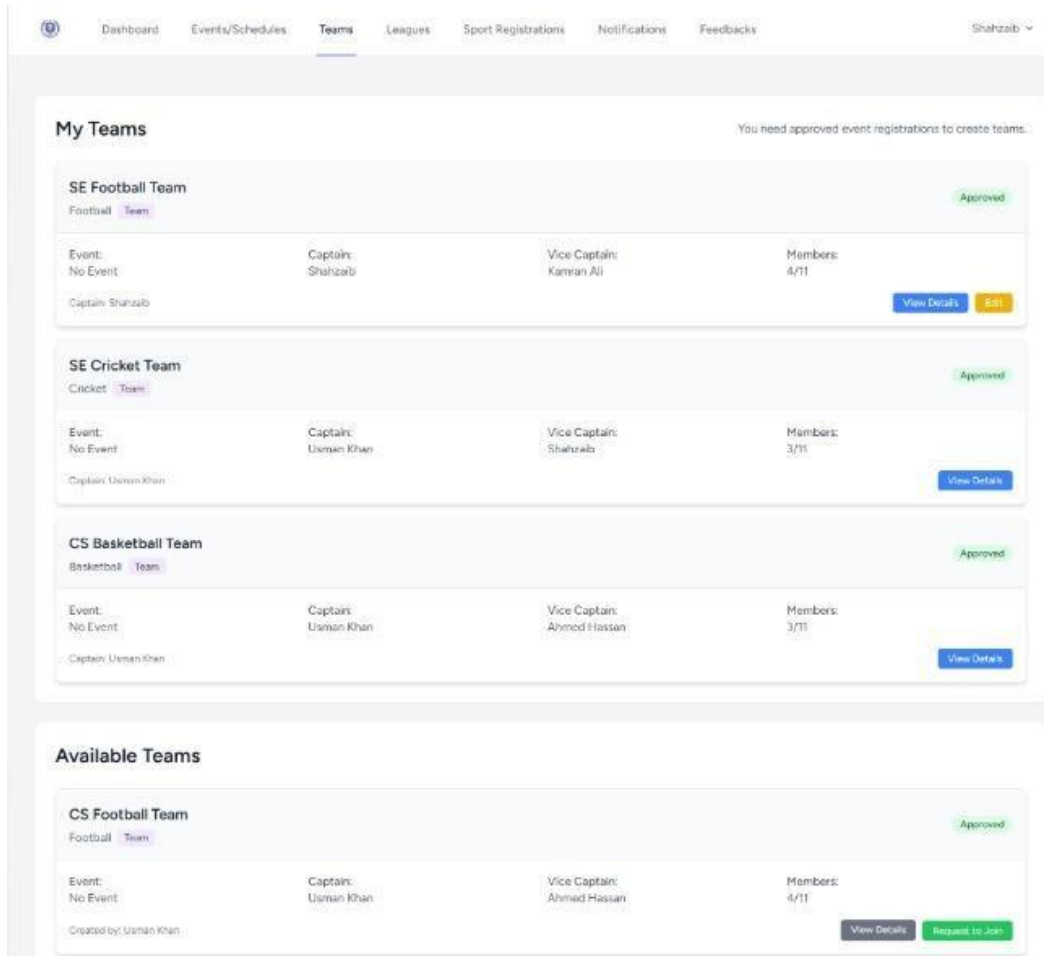


Figure 24 Teams Showcase

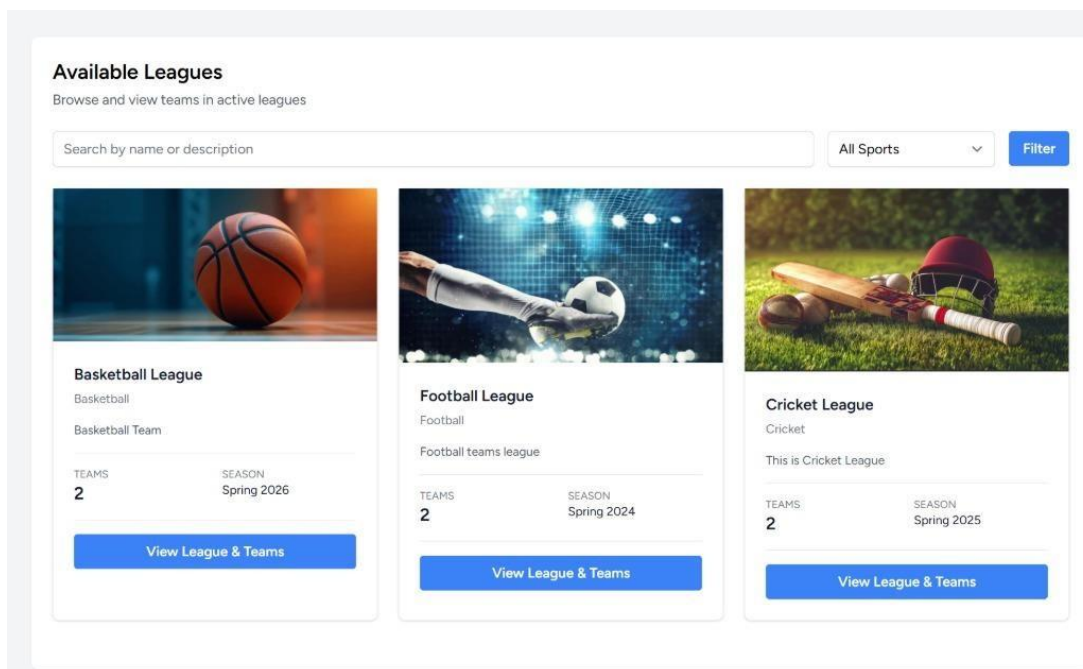


Figure 25 Available Leagues

My Sport Registrations

View and manage your sport registration requests

+ New Registration

SPORT	TYPE	TEAM NAME	STATUS	SUBMITTED	ACTIONS
Badminton	Individual	-	Approved	Apr 28, 2026	
Basketball	Team	CS Basketball Team	Approved	Apr 20, 2026	
Cricket	Team	SE Cricket Team	Approved	Apr 20, 2026	
Football	Team	SE Football Team	Approved	Apr 20, 2026	
Volleyball	Team	-	Pending	Apr 20, 2026	

Figure 26 Sport Registration

My Notifications

Sport Weak

The student weak comes in tommorrow weak

14 hours ago

High priority

Mark Unread

Figure 27 My Notification

My Feedbacks

SUBMIT NEW FEEDBACK

FEEDBACK	SUBMITTED AT	
Hello i am shahzib we win the cricket match	28 Apr 2026, 06:16	View

Figure 28 Feedbacks

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

Testing and evaluation are critical phases in the development of the University Sports Management System (USMS), as they ensure that the application functions correctly, efficiently, and reliably under different conditions. The main purpose of this phase is to identify and remove errors, verify system performance, and ensure that all system functionalities meet the defined requirements. Proper testing helps in improving the overall quality of the system and provides confidence that the application can be used effectively by both administrators and students.

In USMS, testing is performed on different components including the Frontend interfaces, the FastAPI Backend, the MySQL Database, and the AI Microservice housing the Random Forest and Gradient Boosting Regression models. Since the system involves intelligent schedule optimization and performance prediction, it is important to ensure that AI outputs are accurate, consistent, and generated within an acceptable time. Additionally, features such as user authentication, role-based access control, event management, team registration, schedule generation, and feedback submission are all tested to confirm that they work smoothly and securely.

Different testing techniques are applied to evaluate the system from multiple perspectives. Manual testing is used to check system behavior through real user interactions, while automated testing helps in verifying system processes and performance under load. The system is tested across various scenarios to ensure reliability, usability, and consistency.

This chapter presents the testing strategies used in USMS, including manual testing methods such as system testing, unit testing, functional testing, and integration testing. It also discusses automated testing approaches and the tools used to evaluate system performance and AI model accuracy.

6.1 Manual Testing

Manual testing is an important part of the testing process in USMS, where the application is tested by interacting with different features to ensure correct functionality. In this approach, testers perform various actions such as logging in, creating events, generating schedules, registering for teams, and submitting feedback to verify that the system behaves as expected. Manual testing helps in identifying errors, usability issues, and unexpected system behavior that may not be detected through automated testing.

In USMS, manual testing is performed on all major components including the Admin Dashboard, the Student Portal, the FastAPI backend endpoints, the AI microservice, and the MySQL database operations. Each feature is tested by simulating real user interactions to ensure that the system provides accurate results and smooth performance. For example, schedule generation is tested to confirm that the Random Forest model produces conflict-free outputs, and performance prediction is tested to verify that the Gradient Boosting Regression model returns consistent Performance Index scores [4][5].

Manual testing also plays an important role in evaluating user experience. Since USMS is designed for use by university administrators and students, it is necessary to ensure that both interfaces are simple, responsive, and easy to navigate. Testers verify that administrators can manage events, teams, and venues without confusion, and that students can register, view schedules, and submit feedback without requiring technical knowledge [3][6].

Another key aspect of manual testing is error handling. Different test cases are executed to check how the system behaves under invalid inputs or failure conditions. Scenarios such as incorrect login credentials, duplicate team registrations, schedule generation with insufficient data, and unauthorized access attempts are all tested to ensure that the system provides appropriate error messages and handles such situations gracefully.

Overall, manual testing ensures that USMS is reliable, user-friendly, and free from major functional errors. It provides a strong foundation before moving towards more detailed testing such as system testing, unit testing, functional testing, and integration testing.

6.1.1 System Testing

System testing is performed to evaluate the complete and integrated USMS to ensure that all components work together correctly. In this phase, the entire system is tested as a whole including the Frontend, FastAPI Backend, AI Microservice, and MySQL Database to verify that it meets all specified requirements. The main goal of system testing is to identify any issues that may arise when different modules interact with each other [7].

In USMS, system testing is carried out by simulating real-world scenarios where administrators and students interact with the application. This includes testing features such as user login, event creation, team registration, AI schedule generation, performance prediction, notification dispatch,

and feedback submission. The system is also tested for performance, security, and error handling to ensure smooth operation under different conditions.

System testing ensures that the application behaves correctly from the user's perspective and that all integrated components function without errors. It also verifies that the AI models produce accurate outputs and that data is consistently stored and retrieved from the MySQL database throughout all workflows.

Key Areas Covered in System Testing:

- Verification of complete system workflow from login to output
- Validation of user interactions through both Admin Dashboard and Student Portal
- Testing of data flow between Frontend, FastAPI Backend, AI Microservice, and MySQL
- Checking system response under normal and edge case conditions
- Ensuring proper enforcement of role-based access control across all modules

Table 9 System Testing

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-01	Admin Login	Valid admin credentials	Admin redirected to Dashboard	Pass
TC-02	Student Login	Valid student credentials	Student redirected to Portal	Pass
TC-03	Unauthorized Access	Student accessing admin endpoint	Access denied with 403 error	Pass
TC-04	Event Creation	Valid event data entered	Event created and stored in MySQL	Pass
TC-05	Team Creation	Valid team data entered	Team created and visible in system	Pass
TC-06	Venue Creation	Valid venue data entered	Venue created and available for scheduling	Pass
TC-07	AI Schedule Generation	Sufficient teams and venues registered	Conflict-free schedule generated and stored	Pass

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-08	Performance Prediction	Historical data available for team	Performance Index score generated and displayed	Pass
TC-09	Student Team Registration	Open team with available slots	Registration submitted with pending status	Pass
TC-10	System Response	Multiple concurrent requests	System responds without delay or crash	Pass

6.1.2 Unit Testing

Unit testing is a testing technique in which individual components or modules of the system are tested separately to ensure that each part functions correctly. The main purpose of unit testing is to verify that every module of the system, such as authentication, event management, AI inference, and database operations, performs as expected before being integrated into the complete system [7].

In USMS, unit testing is performed on all individual modules including user authentication, event CRUD operations, team management, venue management, schedule generation, performance prediction, registration handling, notification dispatch, and feedback submission. Each module is tested independently to ensure that it produces correct outputs for given inputs, helping to identify errors at an early stage and simplify debugging.

For example, the authentication module is tested to verify JWT token generation and role assignment, while the schedule generation module is tested to ensure that the Random Forest model endpoint returns a properly structured schedule JSON. Similarly, the performance prediction module is tested to confirm that the Gradient Boosting Regression endpoint returns a valid Performance Index score within the expected numerical range [4][5].

Unit testing improves system reliability by ensuring that each component works correctly before integration, reducing the chances of errors in later stages and helping maintain code quality throughout development.

Key Areas Covered in Unit Testing:

- Testing individual FastAPI endpoint modules independently
- Verifying input validation and output structure of each API function
- Ensuring correct behavior of JWT authentication and RBAC middleware
- Validating AI model input preprocessing and output parsing
- Testing all database INSERT, SELECT, UPDATE, and DELETE operations
- Checking notification and feedback module responses

Table 10 Unit Testing

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-11	Login Module	Valid credentials	JWT token returned with correct role	Pass
TC-12	Login Module	Invalid credentials	401 Unauthorized error returned	Pass
TC-13	Event Creation Module	Valid event data	Event record created in MySQL	Pass
TC-14	Event Creation Module	Missing required fields	Validation error returned	Pass
TC-15	Team Creation Module	Valid team data	Team record created successfully	Pass
TC-16	Schedule Generation Module	Valid scheduling parameters	Conflict-free schedule JSON returned	Pass
TC-17	Schedule Generation Module	Insufficient teams or venues	Error message returned	Pass
TC-18	Performance Prediction Module	Valid historical data	Performance Index score returned	Pass
TC-19	Performance Prediction Module	No historical data available	Insufficient data message returned	Pass
TC-20	Registration Module	Valid student and team ID	Registration record created with pending status	Pass

6.1.3 Functional Testing

Functional testing is performed to verify that all functionalities of USMS operate according to the specified requirements defined in Chapter 3. It ensures that features such as user authentication, event management, team and venue management, AI schedule generation, performance prediction, student registration, notification dispatch, and feedback submission all work correctly from the user's perspective [7].

The system is tested with both valid and invalid inputs to ensure proper responses and error handling at every interaction point. Special attention is given to the AI-driven features, where schedule outputs are verified to be conflict-free and performance prediction scores are verified to fall within logical ranges. The accuracy and responsiveness of both the Random Forest and Gradient Boosting Regression models are confirmed through repeated functional test executions [4][5].

Overall, functional testing confirms that the system behaves exactly as expected across all defined functional requirements and fulfills the intended purpose of each module within USMS.

Key Areas Covered in Functional Testing:

- Verification of user registration, login, and role assignment functionality
- Validation of all event, team, and venue management operations
- Testing of AI schedule generation for correctness and conflict-freedom
- Checking performance prediction output for logical consistency
- Verification of student registration and approval workflow
- Testing of notification dispatch and student receipt
- Validation of feedback submission and retrieval

Table 11 Functional Testing

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-21	User Registration	Valid student data	Account created and stored in MySQL	Pass
TC-22	Admin Login	Correct credentials	JWT token issued, redirected to Dashboard	Pass

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-23	Admin Login	Incorrect credentials	Error message displayed	Pass
TC-24	Event Management	Create, update, delete event	All operations reflect correctly in database	Pass
TC-25	Team Registration	Student submits registration	Pending registration stored in database	Pass
TC-26	Admin Approves Registration	Pending registration selected	Status updated to approved, student notified	Pass
TC-27	AI Schedule Generation	Admin triggers generation	Conflict-free schedule generated and displayed	Pass
TC-28	Performance Prediction	Admin selects team	Performance Index score displayed in report	Pass
TC-29	Notification Dispatch	Admin sends notification	Notification stored and visible on Student Portal	Pass
TC-30	Feedback Submission	Student submits feedback for completed event	Feedback stored and accessible to admin	Pass

6.1.4 Integration Testing

Integration testing is performed to verify that different modules of USMS work together correctly after being combined. While unit testing focuses on individual components, integration testing ensures that the interaction between these components is smooth and error-free. The main goal is to identify issues that may occur when different parts of the system communicate with each other [7].

In USMS, integration testing is carried out by combining modules such as the Frontend, FastAPI Backend, AI Microservice, and MySQL Database. The system is tested to ensure that data flows correctly between all components. For example, when an administrator triggers schedule generation, the frontend sends a request to the FastAPI backend, which retrieves data from

MySQL, forwards it to the AI microservice, receives the optimized schedule, stores it in MySQL, and returns it to the frontend for display. This entire flow is tested end-to-end to confirm seamless integration [1][4].

Integration testing also verifies the interaction between the FastAPI backend and the AI microservice for both the Random Forest schedule optimization and Gradient Boosting Regression performance prediction workflows. These integrations are tested to ensure that model inputs are correctly formatted, model outputs are correctly parsed, and all results are accurately persisted in MySQL [2][5].

Another important aspect is ensuring data consistency across modules. Performance Index scores generated by the AI model must be correctly stored in MySQL and accurately retrieved when performance reports are accessed. User authentication must function correctly and consistently across all frontend requests and backend validations.

Key Areas Covered in Integration Testing:

- Integration between Frontend and FastAPI Backend
- Communication between FastAPI Backend and AI Microservice
- Data flow between Backend and MySQL Database
- End-to-end schedule generation workflow validation
- End-to-end performance prediction workflow validation
- Student registration and admin approval cross-module workflow
- Notification dispatch and student receipt cross-module workflow

Table 12 Integration Testing

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-31	Frontend–Backend Integration	Admin submits login form	JWT token issued and dashboard loaded	Pass
TC-32	Backend–MySQL Integration	Event creation request	Event record stored in MySQL correctly	Pass
TC-33	Backend–AI Integration (Schedule)	Schedule generation triggered	Random Forest model returns valid schedule	Pass

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-34	Backend–AI Integration (Prediction)	Performance prediction triggered	Gradient Boosting model returns Performance Index	Pass
TC-35	AI–MySQL Integration	Schedule result received	Schedule records stored in MySQL correctly	Pass
TC-36	Registration Workflow	Student submits, admin approves	Status updated and notification sent correctly	Pass
TC-37	End-to-End System Workflow	Full system operation	All components work together without errors	Pass

6.2 Automated Testing

Automated testing is a testing approach in which predefined test cases are executed using automated tools and scripts without manual intervention. This type of testing is used to improve efficiency, reduce human error, and ensure consistent system performance. Automated testing is especially useful for repetitive tasks, API validation, and performance evaluation under load [7]. In USMS, automated testing is applied to verify FastAPI backend endpoints, AI model responses, and system performance under concurrent usage conditions. Since the system involves machine learning inference and real-time data processing, automated testing helps ensure that the system responds quickly and consistently across multiple simultaneous user requests. Automated testing also validates that API responses maintain correct structure, status codes, and data integrity across all endpoints.

6.2.1 API Testing

API testing is performed to verify that all FastAPI backend endpoints of USMS are working correctly. Each endpoint is tested to ensure it returns correct responses, enforces authentication, applies role-based access control, and handles both valid and invalid requests appropriately. Since USMS relies entirely on API communication between the frontend and backend, it is essential that all endpoints function reliably under different conditions [1].

During API testing, each endpoint is tested using Postman with different types of inputs including valid data, missing fields, invalid tokens, and unauthorized roles to validate behavior comprehensively. Special attention is given to the AI endpoints — schedule generation and performance prediction — to ensure that they correctly receive parameters, invoke the appropriate model, and return properly structured JSON responses.

Key Areas Covered in API Testing:

- All CRUD endpoints tested for correct input validation and response structure
- Authentication endpoints tested for token generation and expiry handling
- RBAC enforcement tested by sending admin-role requests with student tokens
- AI schedule generation endpoint tested for correct JSON output format
- AI performance prediction endpoint tested for numerical score validity
- Error handling tested for all anticipated failure scenarios

6.2.2 Performance Testing

Performance testing is conducted to evaluate how USMS behaves under different workloads and usage conditions. This type of testing ensures that the system can handle multiple concurrent users efficiently without performance degradation. In USMS, performance testing focuses on measuring response time, system stability, and scalability across both standard operations and AI-driven operations [1][7].

Since the application involves machine learning inference for schedule optimization and performance prediction, it is important to ensure that AI results are generated within the defined five-second maximum response time requirement. Various scenarios are tested including multiple simultaneous API requests, consecutive AI prediction calls, and sustained database read and write operations under load.

Key Areas Covered in Performance Testing:

- System response time measured for all standard CRUD operations
- AI schedule generation response time verified against five-second maximum requirement
- AI performance prediction response time verified for multiple teams simultaneously
- Multiple concurrent user sessions simulated to test FastAPI asynchronous handling
- MySQL query performance evaluated under increasing data volumes
- System stability tested under continuous and sustained usage conditions

6.2.3 Model Testing

Model testing focuses on evaluating the performance and accuracy of both AI models used in USMS — the Random Forest schedule optimization model and the Gradient Boosting Regression performance prediction model. Each model is tested using held-out test datasets to ensure that it provides reliable and consistent outputs before being deployed in the live system [4][5].

The Random Forest model is evaluated using scheduling accuracy metrics, measuring the percentage of generated schedules that are completely free of venue and team conflicts. The Gradient Boosting Regression model is evaluated using Mean Absolute Error (MAE) and R² score metrics on the test dataset, with a minimum acceptable R² score of 0.80 required for deployment approval. Both models are also tested for inference speed to ensure compliance with the defined response time requirements.

Key Areas Covered in Model Testing:

- Random Forest model evaluated for conflict-free scheduling accuracy
- Gradient Boosting Regression model evaluated for MAE and R² score
- Both models tested for inference speed under single and batch prediction requests
- Model consistency verified across repeated calls with identical inputs
- Edge cases tested including minimum data inputs and boundary constraint scenarios
- Model retraining pipeline tested to confirm correct model persistence and loading

Table 13 Automated Testing

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-38	API Response — Login	Valid credentials	JWT token returned with 200 status	Pass
TC-39	API Error Handling	Invalid token on protected endpoint	401 Unauthorized returned	Pass
TC-40	API RBAC Enforcement	Student token on admin endpoint	403 Forbidden returned	Pass
TC-41	Performance Test — CRUD	Multiple concurrent requests	All requests processed within two seconds	Pass

Test Case ID	Test Case	Attribute Value	Expected Result	Result
TC-42	Performance Test — AI Schedule	Schedule generation under load	Response returned within five seconds	Pass
TC-43	Performance Test — AI Prediction	Prediction for multiple teams	All scores returned within five seconds	Pass
TC-44	Model Test — Random Forest	Test scheduling dataset	Conflict-free accuracy above 85%	Pass
TC-45	Model Test — Gradient Boosting	Test performance dataset	R ² score above 0.80, MAE documented	Pass
TC-46	System Test Load	Sustained concurrent usage	System remains stable without errors	Pass

6.3 Tools Used for Testing

Various tools and technologies are used during the testing phase of USMS to ensure that all functionalities work correctly and efficiently. These tools help in identifying errors, validating system performance, and ensuring smooth interaction between different components of the system. Since the project involves multiple technologies including frontend interfaces, a FastAPI backend, an AI microservice, and a MySQL database, different tools are used for testing each part of the system.

API testing tools are used to verify backend functionality and ensure that all endpoints respond correctly with the expected status codes, response structures, and data payloads. Browser-based tools are used to test the user interface and check system behavior from the user's perspective across different device types. Machine learning evaluation tools are used to measure the performance and accuracy of both AI models. Database tools are used to verify data storage, retrieval, and integrity operations within MySQL.

These tools play an important role in improving system reliability, performance, and usability. They also help developers detect issues at an early stage and ensure that the system meets all defined requirements before final deployment.

Table 14 Tools Used for Testing

Tool	Purpose	Usage in USMS
Postman	API Testing	Used to test all FastAPI endpoints and verify request/response handling, authentication, and RBAC enforcement
Google Chrome DevTools	Frontend Testing	Used for manual testing of UI responsiveness, network request inspection, and console error debugging
FastAPI Swagger UI	API Documentation and Testing	Used to test all API endpoints directly through the browser-based OpenAPI interface
Scikit-learn Metrics	AI Model Testing	Used to evaluate Random Forest scheduling accuracy and Gradient Boosting Regression MAE and R ² score
MySQL Workbench	Database Testing	Used to verify data storage, retrieval, foreign key integrity, and query performance in MySQL
VS Code Debugger	Unit and Integration Testing	Used to debug FastAPI backend modules and trace errors during unit and integration test execution

Chapter 7

Conclusion and Future Work

7. Conclusion

The University Sports Management System (USMS) is a web-based application developed to improve and automate the management of sports activities within a university environment. Traditional manual methods often create problems such as scheduling conflicts, delayed communication, and difficulty in maintaining records. This project addresses these issues by providing a centralized platform that simplifies and organizes all sports-related operations in an efficient manner.

The system offers separate access for administrators and students, ensuring smooth coordination between management and participants. A major feature of USMS is the integration of artificial intelligence to support intelligent decision-making. The system uses Random Forest and Gradient Boosting Regression models for tasks such as schedule optimization and team performance prediction. These models analyze available data to produce accurate and efficient results, reducing manual effort and improving overall system performance.

To ensure a complete and user-friendly solution, the frontend of the system is developed using Bootstrap, Tailwind CSS, and JavaScript to provide a responsive interface. The backend is built with FastAPI, which manages business logic, API communication, and integration with machine learning services. MySQL is used as the database to store and manage data in a structured and reliable way.

The development process followed the Agile Software Development Model, utilizing sprint-based iterative approaches to manage evolving AI requirements and parallel development. The testing phase—which included manual testing, system testing, unit testing, functional testing, integration testing, and automated testing—ensured that all system functionalities operate efficiently and provide consistent results.

In conclusion, the University Sports Management System provides a modern, scalable, and intelligent solution for university sports management. By combining web technologies with machine learning, the system improves efficiency, enhances user experience, and supports better decision-making.

7.1 Future Work

Although the USMS successfully achieves its current objectives, it offers a practical foundation for future enhancements such as mobile support, real-time notifications, and advanced analytics. Future work can focus on improving system accessibility, extending functionality, and increasing scalability to support a larger user base.

One of the primary areas for improvement is the development of a dedicated mobile application. A mobile version would significantly improve accessibility, allowing students and administrators to interact with the system more conveniently on the go, facilitating quicker team registrations and schedule checks.

Integration of real-time notifications is another important direction. Implementing SMS or mobile push notifications for schedule updates, event changes, and registration approvals would enhance communication and ensure users are instantly informed without needing to manually check the portal.

The system can also be enhanced by incorporating advanced analytics. Visual dashboards with deeper insights into player statistics, tournament progress, and venue utilization could provide administrators with more comprehensive decision-making tools. Furthermore, expanding the AI capabilities to include player injury prediction or live match tracking could transform the system into an even more comprehensive sports management solution.

Finally, cloud-based scalability can be further improved to support multi-university integration. Optimizing the system to manage inter-university sports events and larger datasets efficiently would broaden its real-world applicability.

Table 15 Proposed Future Enhancements

Feature	Description	Expected Benefit
Mobile Application	Develop an Android/iOS application.	Increased accessibility and convenience for users on the go.
Real-Time Notifications	Implement SMS and push notifications for updates.	Improved communication and instant updates for schedules and events.
Advanced Analytics	Add visual dashboards for player and venue statistics.	Better data-driven decision-making and reporting for administrators.

Feature	Description	Expected Benefit
Expanded Capabilities AI	Introduce injury prediction and live tracking.	Enhanced team management, safety, and real-time performance insights.
Multi-University Integration	Scale the system to manage inter-university events.	Broader usage and support for larger-scale event coordination.

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson, 2016.
- [2] FastAPI, “FastAPI,” [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Apr. 28, 2026].
- [3] MySQL, “MySQL Documentation,” [Online]. Available: <https://dev.mysql.com/doc/>. [Accessed: Apr. 28, 2026].
- [4] Bootstrap, “Bootstrap Documentation,” [Online]. Available: <https://getbootstrap.com/docs/>. [Accessed: Apr. 28, 2026].
- [5] Tailwind CSS, “Tailwind CSS Documentation,” [Online]. Available: <https://tailwindcss.com/docs/>. [Accessed: Apr. 28, 2026].
- [6] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.